二维动态避障路径规划中的 Loss 函数

——原理、代码实现与权重敏感性实验

实验代码： `Imperative_learning_2D_moving.py`
实验环境： Python 3.13、PyTorch 2.13.0（CPU）
实验日期： 2026-08-01

摘要

本项目使用 PyTorch 的自动求导能力，直接优化二维机器人的未来控制序列，使机器人从起点运动到目标点，同时避开四个动态障碍物。规划器的核心是由目标损失、避障损失、控制损失和平滑损失组成的加权总 Loss。

本文首先解释四项 Loss 的公式、物理含义以及它们如何通过梯度改变机器人路径；随后以当前代码的权重 `(100, 100, 0.10, 3)` 为基准，进行 9 组单变量实验。每组实验只改变一个权重，并保持机器人初始状态、障碍物运动、优化次数和学习率不变。实验共生成 27 张图，用于比较轨迹、收敛过程、安全距离和控制平滑性。

实验表明：目标项决定机器人是否到达终点；避障项是防止碰撞的必要条件；当前 `0.10` 的控制权重和 `3` 的平滑权重作用较弱；权重过大时会牺牲其他目标。同时，由于问题是非凸优化，权重与最终结果之间不一定具有简单的单调关系。

---

1. 项目在解决什么问题

机器人起点和目标点为：

$$\mathbf{p}_0 = (-2.5, -1.5), \qquad \mathbf{p}_g = (2.5, 1.5)$$

代码规划未来 70 个时间步，每一步为：

$$\Delta t = 0.1\ \mathrm{s}$$

所以规划时域为：

$$T = 70 \times 0.1 = 7\ \mathrm{s}$$

需要被优化的是未来 70 步二维控制量：

$$\mathbf{u}_t = (u_{x,t}, u_{y,t}), \qquad t = 0, 1, \ldots, 69$$

一共包含：

$$70 \times 2 = 140$$

个待优化变量。机器人采用简单的速度积分模型：

$$\mathbf{p}_{t+1} = \mathbf{p}_t + \Delta t\, \mathbf{u}_t$$

代码没有训练神经网络，而是把 `raw_controls` 本身作为可训练参数。实际控制量为：

$$\mathbf{u}_t = v_{\max} \tanh(\mathbf{a}_t)$$

其中 `raw_controls` 对应 $\mathbf{a}_t$，且 $v_{\max} = 1.0\ \mathrm{m/s}$。`tanh` 把每个速度分量限制在 $[-1, 1]$ 之间。

> 注意：限制的是 u_x 和 u_y 两个分量，而不是合速度。因此合速度 $\sqrt{u_x^2 + u_y^2}$ 理论上可以达到 $\sqrt{2} \approx 1.414\ \mathrm{m/s}$。

---

2. 总 Loss 的组成

当前实际代码第 143 行使用：

```
total_loss = (
    100.0 * goal_loss
    + 100.0 * obstacle_loss
    + 0.10 * control_loss
    + 3.00 * smoothness_loss
)
```

对应公式为：

$$\boxed{L_{\text{total}} = 100 L_{\text{goal}} + 100 L_{\text{obstacle}} + 0.10 L_{\text{control}} + 3 L_{\text{smooth}}}$$

一般形式可以写成：

$$L_{\text{total}} = w_g L_{\text{goal}} + w_o L_{\text{obstacle}} + w_u L_{\text{control}} + w_s L_{\text{smooth}}$$

四个权重不是唯一的数学答案，而是把任务要求转换成优化优先级的超参数。真正影响总 Loss 的是 $w_i L_i$，不能只比较权重数字的大小。

---

3. 四项 Loss 的公式与含义

3.1 目标损失 Goal Loss

代码为：

```
goal_loss = torch.sum((positions[-1] - GOAL) ** 2)
```

公式为：

$$\boxed{L_{\text{goal}} = \|\mathbf{p}_T - \mathbf{p}_g\|^2}$$

展开后为：

$$L_{\text{goal}} = (x_T - x_g)^2 + (y_T - y_g)^2$$

它只检查最后一个时刻的位置。机器人越接近目标，L_{goal} 越小；完全到达目标时该项为 0。

例如机器人最后位于 $(2.3, 1.4)$，目标为 $(2.5, 1.5)$：

$$L_{\text{goal}} = (2.3 - 2.5)^2 + (1.4 - 1.5)^2 = 0.05$$

若目标权重为 100，则它对总 Loss 的贡献是：

$$100 \times 0.05 = 5$$

3.2 避障损失 Obstacle Loss

代码使用 181 条 LiDAR 射线测量每个时刻与最近障碍物表面的距离 d_t：

```
penetration = F.relu(LIDAR_SAFE_DISTANCE - distances)
obstacle_loss = torch.sum(penetration ** 2)
```

公式为：

$$\boxed{L_{\text{obstacle}} = \sum_{t=0}^{T} [\max(0, d_{\text{safe}} - d_t)]^2}$$

安全距离由机器人半径和额外安全余量组成：

$$d_{\text{safe}} = R_{\text{robot}} + d_{\text{margin}} = 0.15 + 0.15 = 0.30 \text{ m}$$

当 $d_t \geq 0.30$ m 时，该时刻的惩罚为 0；当距离小于 0.30 m 时，才产生惩罚和避障梯度。

例如某一时刻 $d_t = 0.20$ m：

$$L_{\text{obstacle},t} = (0.30 - 0.20)^2 = 0.01$$

如果连续 10 个时刻都保持该距离，则原始避障损失为：

$$10 \times 0.01 = 0.10$$

因此该项同时反映"进入危险区域有多深"和"危险持续了多久"。理想值为：

$$\boxed{L_{\text{obstacle}} = 0}$$

但 `Obstacle Loss = 0` 只表示没有低于 0.30 m，不代表障碍距离越大越好。超过安全线后，该项不再提供继续远离障碍物的梯度。

3.3 控制损失 Control Loss

代码为：

```
control_loss = torch.mean(torch.sum(controls ** 2, dim=1))
```

公式为：

$$\boxed{L_{\text{control}} = \frac{1}{T}\sum_{t=0}^{T-1}\|\mathbf{u}_t\|^2}$$

展开后为：

$$L_{\text{control}} = \frac{1}{T}\sum_{t=0}^{T-1}(u_{x,t}^2 + u_{y,t}^2)$$

该项鼓励机器人减少不必要的高速运动。若权重过大，优化器会更偏向低速甚至停止，因此当前代码只给了较小的权重 `0.10`。

3.4 平滑损失 Smoothness Loss

代码为：

```
control_changes = controls[1:] - controls[:-1]
smoothness_loss = torch.mean(
    torch.sum(control_changes ** 2, dim=1)
)
```

公式为：

$$\boxed{L_{\text{smooth}} = \frac{1}{T-1}\sum_{t=0}^{T-2}\|\mathbf{u}_{t+1} - \mathbf{u}_t\|^2}$$

它衡量相邻两个时刻的控制变化。若速度从 $(1, 0)$ 突然变成 $(-1, 0)$，单步平滑代价为：

$$\|(-1, 0) - (1, 0)\|^2 = 4$$

如果速度只从 $(0.5, 0)$ 变成 $(0.6, 0.1)$，单步代价为：

$$0.1^2 + 0.1^2 = 0.02$$

所以该项能够抑制速度突变和轨迹抖动。但权重过大也会使机器人不愿快速转向，降低避障灵活性。

---

4. Loss 如何改变路径

一次完整计算过程是：

$$\text{raw controls} \rightarrow \text{实际速度} \rightarrow \text{机器人轨迹} \rightarrow \text{LiDAR距离} \rightarrow L_{\text{total}}$$

PyTorch 通过反向传播计算：

$$\frac{\partial L_{\text{total}}}{\partial \mathbf{a}_t}$$

Adam 优化器再根据梯度修改全部 70 步控制序列。用简化的梯度下降形式可表示为：

$$\mathbf{a}^{(k+1)} = \mathbf{a}^{(k)} - \alpha \nabla_{\mathbf{a}} L_{\text{total}}$$

其中学习率为：

$$\alpha = 0.05$$

由于位置通过速度累积得到，较早的控制量会影响后面全部位置。因此优化器修改的不是一个点，而是整条未来轨迹。

---

5. 三类输出图应该怎样看

以下三类图在每组实验中使用相同布局，因此只需先理解一次。

图 A：训练过程图 `01_training_progress.png`

- 左侧显示第 `0、17、34、51、69` 次迭代产生的轨迹，用来观察轨迹如何逐渐改变。
- 右侧显示 Loss 随迭代次数的变化。
- 黑线 `Total` 是加权总 Loss；其余彩色曲线是未加权的原始 Loss。
- 纵轴是对数坐标。Obstacle 曲线显示为 10^{-10} 时，实际值通常已经是 0；使用 10^{-10} 只是因为对数坐标不能显示 0。
- 曲线不必每一步都下降。Adam、动态障碍物和各项 Loss 的竞争都可能造成局部波动，关键是观察整体趋势和最终状态。

图 B：四联评估图 `02_evaluation_dashboard.png`

- **左上：最终轨迹。** 判断机器人是否到达目标、是否绕开动态障碍物。
- **右上：Loss 曲线。** 判断优化是否收敛，以及 Goal、Obstacle、Smooth 项如何变化。
- **左下：安全距离曲线。** 紫线是最近 LiDAR 距离；橙色虚线是 $0.30\ \mathrm{m}$ 安全线；红线是 $0.15\ \mathrm{m}$ 接触线。紫线低于橙线表示违反安全要求，低于红线表示机器人实体已经接触障碍物。
- **右下：控制曲线。** 蓝线和橙线分别为 u_x、u_y，绿色虚线为合速度。黑色点线是单个速度分量上限，不是合速度上限。

这是判断实验是否成功的核心图。判断顺序应为：先看是否碰撞，再看是否到达，最后比较速度、平滑性和路径长度。

图 C：最终场景图 `03_final_scene.png`

- 它对应原程序动态动画的最后一帧。
- 蓝色曲线为机器人路径，彩色虚线和圆形为动态障碍物轨迹与最终位置。
- 红点是最终时刻 LiDAR 射线打到障碍物表面的位置。
- 标题直接给出最终目标误差和整段轨迹的最小 LiDAR 距离。

PDF 不能保留 Matplotlib 窗口的实时运动，因此本文把动画的最终帧保存成静态图。它适合说明最终空间关系；完整动态过程仍应在 VS Code 中运行原程序观看。

---

6. 实验设计

为了保证公平性，所有实验均采用：

- 相同起点、目标点和四个动态障碍物；
- 相同的初始控制序列；

- `DT = 0.1 s`;
- `NUM_STEPS = 70`;
- `TRAINING_ITERATIONS = 70`;
- `LEARNING_RATE = 0.05`;
- 每次只修改一个 Loss 权重。

9 组实验如下：

实验	Goal	Obstacle	Control	Smooth	目的
基准组	100	100	0.10	3	当前代码
移除目标项	0	100	0.10	3	验证目标项是否必要
提高目标权重	500	100	0.10	3	强调终点精度
降低避障权重	100	10	0.10	3	检查安全风险
提高避障权重	100	800	0.10	3	强化避障优先级
移除控制项	100	100	0	3	检查原控制项的影响
强控制约束	100	100	100	3	明显强调低控制能量
移除平滑项	100	100	0.10	0	检查原平滑项的影响
强平滑约束	100	100	0.10	3000	明显强调控制连续性

`Control = 100` 和 `Smooth = 3000` 是敏感性测试，而不是推荐值。之所以需要较大的倍率，是因为原始 Control Loss 约为 10^{0}，而原始 Smooth Loss 只有约 10^{-3}。如果只增加 2 倍或 10 倍，轨迹变化很难在图中看出来。

6.1 评价指标

最终目标误差：

$$e_g = \|\mathbf{p}_T - \mathbf{p}_g\| = \sqrt{L_{\text{goal}}}$$

最小安全余量：

$$c_{\min} = d_{\min} - 0.30$$

- $c_{\min} > 0$：满足安全要求；
- $c_{\min} < 0$：至少一个时刻进入安全区；
- $d_{\min} < 0.15\ \text{m}$：出现实体接触。

此外还记录路径长度、平均速度以及相邻控制量变化的 RMS 指标：

$$\Delta u_{\text{RMS}} = \sqrt{\frac{1}{T-1}\sum_{t} \|\mathbf{u}_{t+1} - \mathbf{u}_t\|^2}$$

---

7. 实验结果汇总

7.1 物理指标

实验	目标误差/m	最小 LiDAR/m	安全余量/m	低于安全线步数	接触步数	路径长度/m	平均速度/(m/s)	Δu_{RMS}
基准组	0.0268	0.6241	+0.3241	0	0	6.3468	0.9067	0.03377
移除目标项	3.1538	0.3856	+0.0856	0	0	4.2428	0.6061	0.03465
目标权重 500	**0.0059**	0.4167	+0.1167	0	0	6.1572	0.8796	0.03131
避障权重 10	0.0450	**0.0242**	**-0.2758**	10	**6**	5.9665	0.8524	0.02652
避障权重 800	0.0136	0.3412	+0.0412	0	0	6.8562	0.9795	0.05904
移除控制项	0.0268	0.6243	+0.3243	0	0	6.3472	0.9067	0.03378
控制权重 100	0.0853	0.3858	+0.0858	0	0	5.9824	0.8546	0.02553
移除平滑项	0.0268	0.6241	+0.3241	0	0	6.3469	0.9067	0.03396
平滑权重 3000	0.0307	0.5847	+0.2847	0	0	6.3133	0.9019	**0.02221**

7.2 原始 Loss 指标

实验	Goal Loss	Obstacle Loss	Control Loss	Smooth Loss	Total Loss*
基准组	7.168e-4	0	0.8398	1.140e-3	0.1591
移除目标项	9.9463	0	0.4882	1.201e-3	0.0524
目标权重 500	3.522e-5	0	0.7938	9.804e-4	0.0999
避障权重 10	2.029e-3	0.3998	0.7435	7.032e-4	4.2773
避障权重 800	1.837e-4	0	0.9682	3.485e-3	0.1256
移除控制项	7.208e-4	0	0.8399	1.141e-3	0.0755
控制权重 100	7.278e-3	0	0.7400	6.519e-4	74.7300
移除平滑项	7.162e-4	0	0.8398	1.153e-3	0.1556
平滑权重 3000	9.420e-4	0	0.8278	4.933e-4	1.6567

* 不同权重组之间不能直接用 Total Loss 的大小排名，因为计算 Total Loss 的公式已经改变。例如移除 Goal 项后 Total Loss 只有 `0.0524`，看似低于基准组，但机器人距离目标还有 `3.1538 m`，实际任务明显失败。

8. 各组实验图与分析

8.1 实验一：基准组 `(100, 100, 0.10, 3)`

基准组最终目标误差为 `0.0268 m`，最小 LiDAR 距离为 `0.6241 m`，比安全线多 `0.3241 m`。整段轨迹没有进入安全区，也没有接触障碍物。Obstacle Loss 最终为 0，说明基准权重能够在当前环境下同时实现到达和避障。

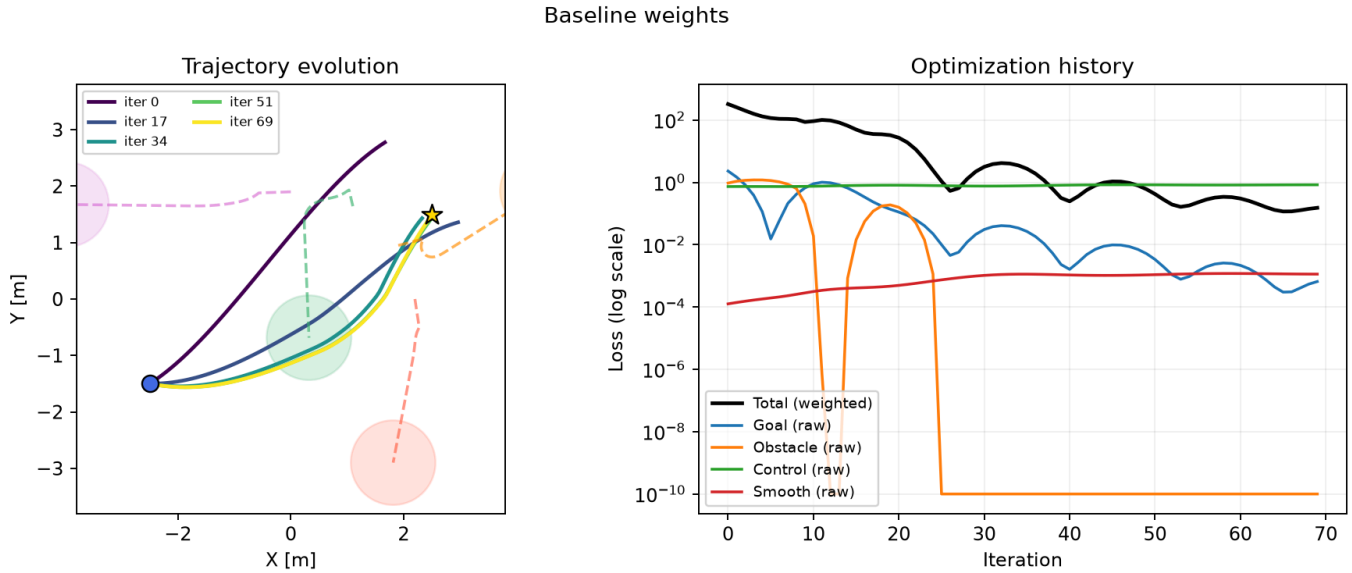


图 8-1：基准组的轨迹演化和 Loss 收敛过程。

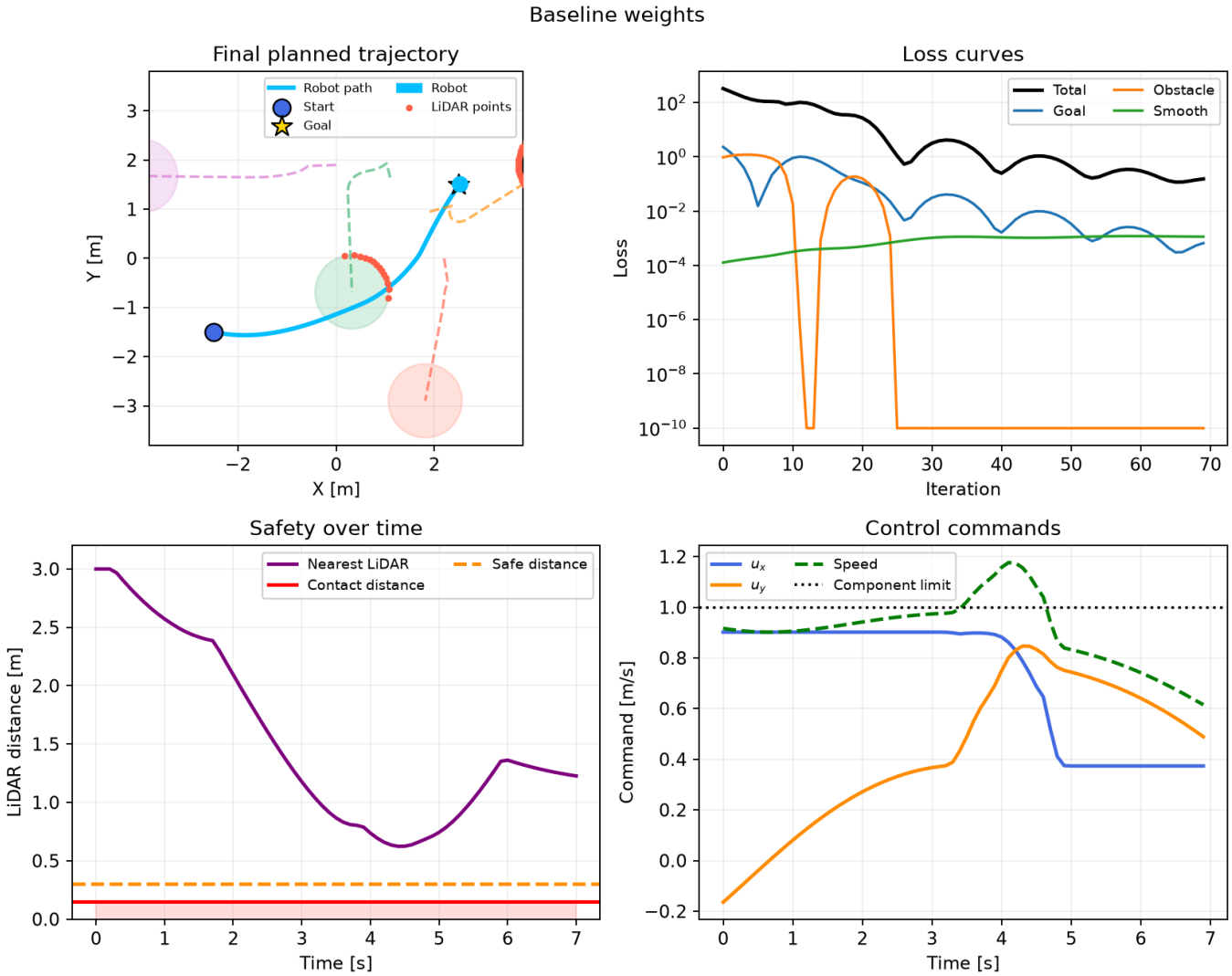


图 8-2：基准组的最终轨迹、Loss、安全距离和控制曲线。

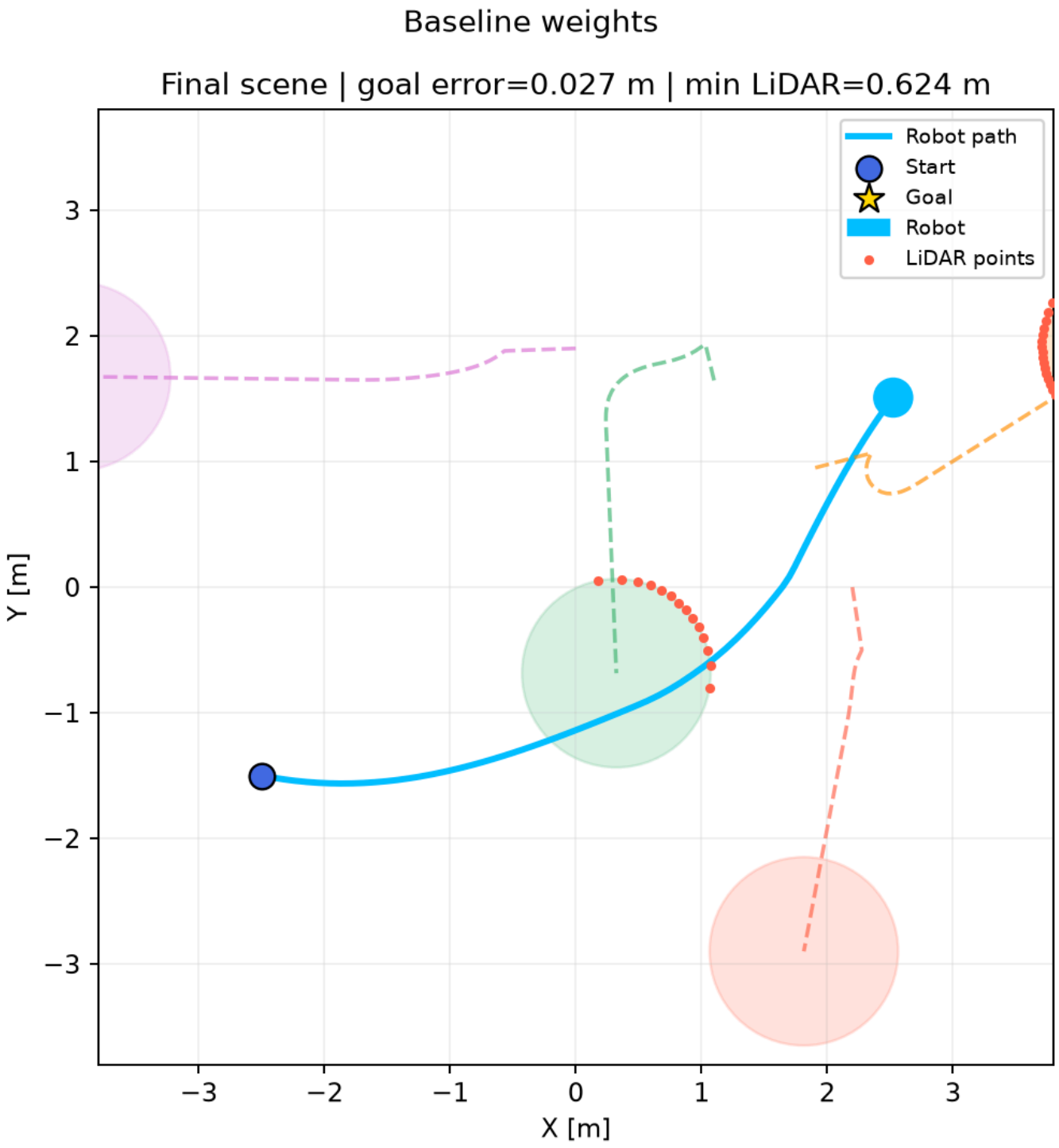


图 8-3：基准组动态动画的最终帧。

结论：基准组综合表现较好，适合作为其他组的比较对象。

8.2 实验二：移除目标损失 `(0, 100, 0.10, 3)`

移除目标项后，最终目标误差增加到 `3.1538 m`。机器人仍然安全，但只运动了约 `4.2428 m`，没有完成到达任务。此时 Total Loss 反而降至 `0.0524`，直接证明：跨权重实验不能仅凭 Total Loss 的数字判断结果好坏。

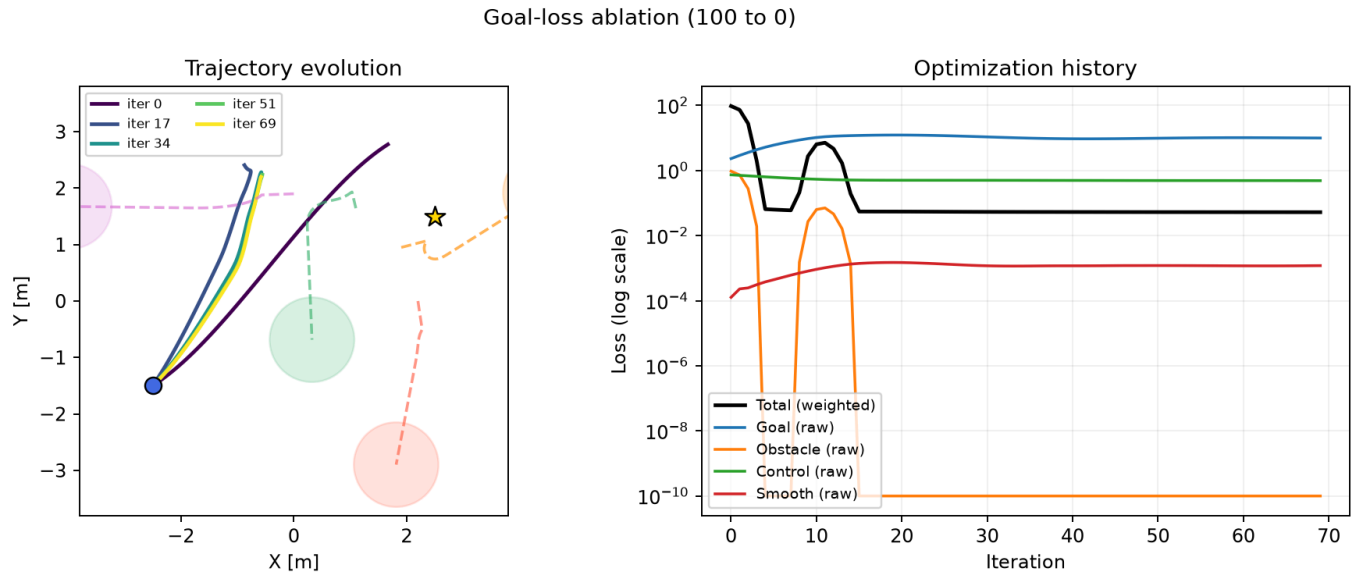


图 8-4：移除目标项后，轨迹不再受到终点吸引。

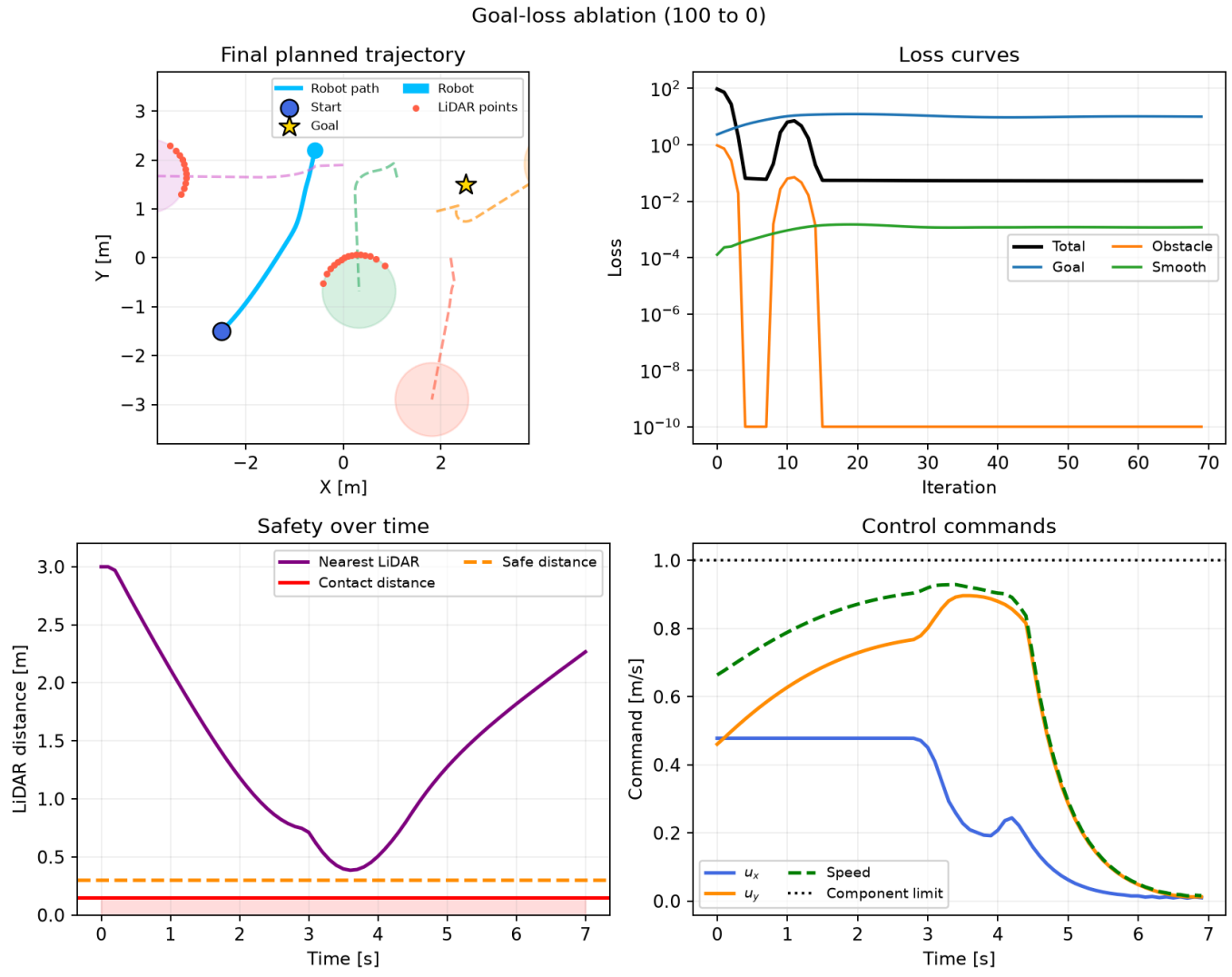


图 8-5：虽然其他损失较小，但机器人未到达目标。

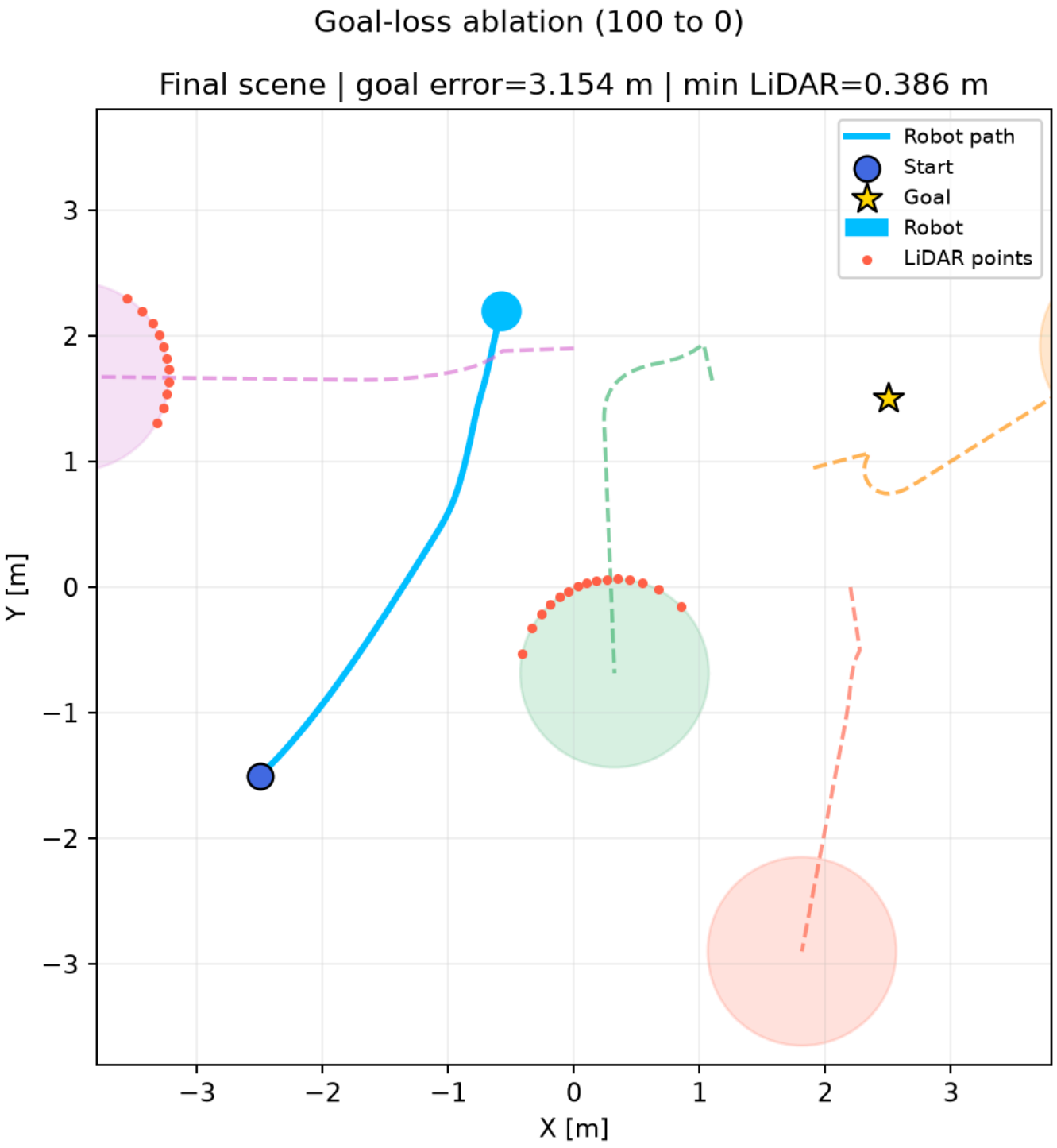


图 8-6：最终位置与目标点相距约 3.15 m。

结论：Goal Loss 是完成导航任务的必要项。

8.3 实验三：目标权重提高到 500

目标权重提高后，目标误差从基准组的 `0.0268 m` 降至 `0.0059 m`，终点精度明显提高。最小 LiDAR 距离为 `0.4167 m`，仍满足安全要求，但安全余量从 `0.3241 m` 缩小到 `0.1167 m`。

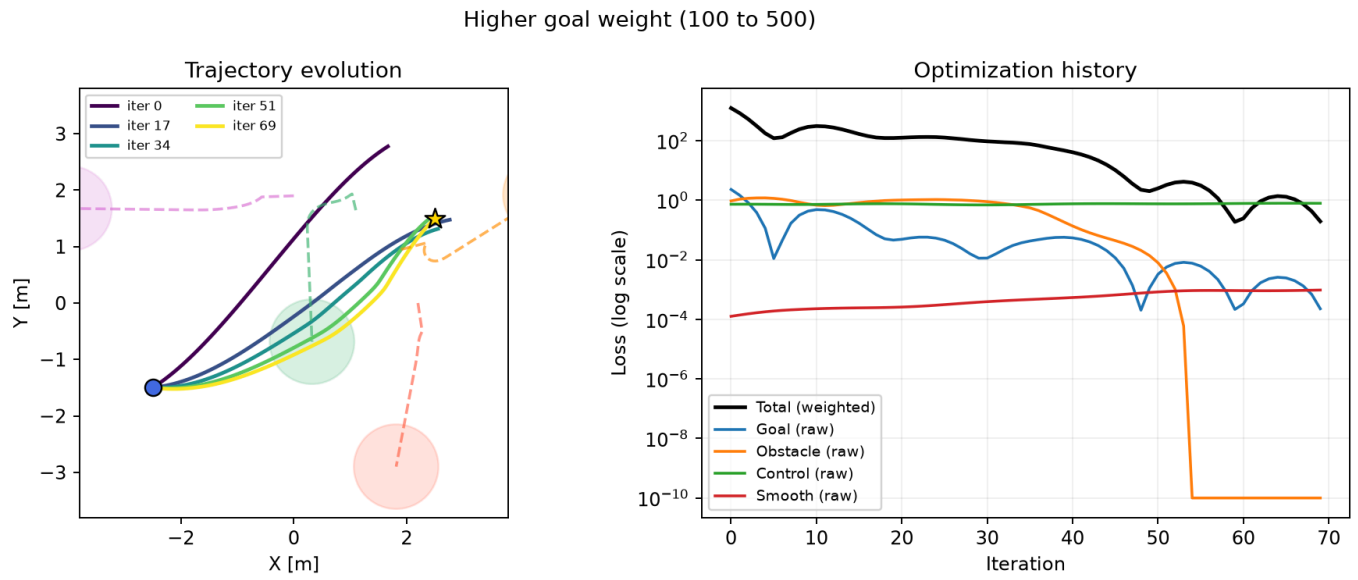


图 8-7：更大的目标权重使优化器更强烈地修正终点误差。

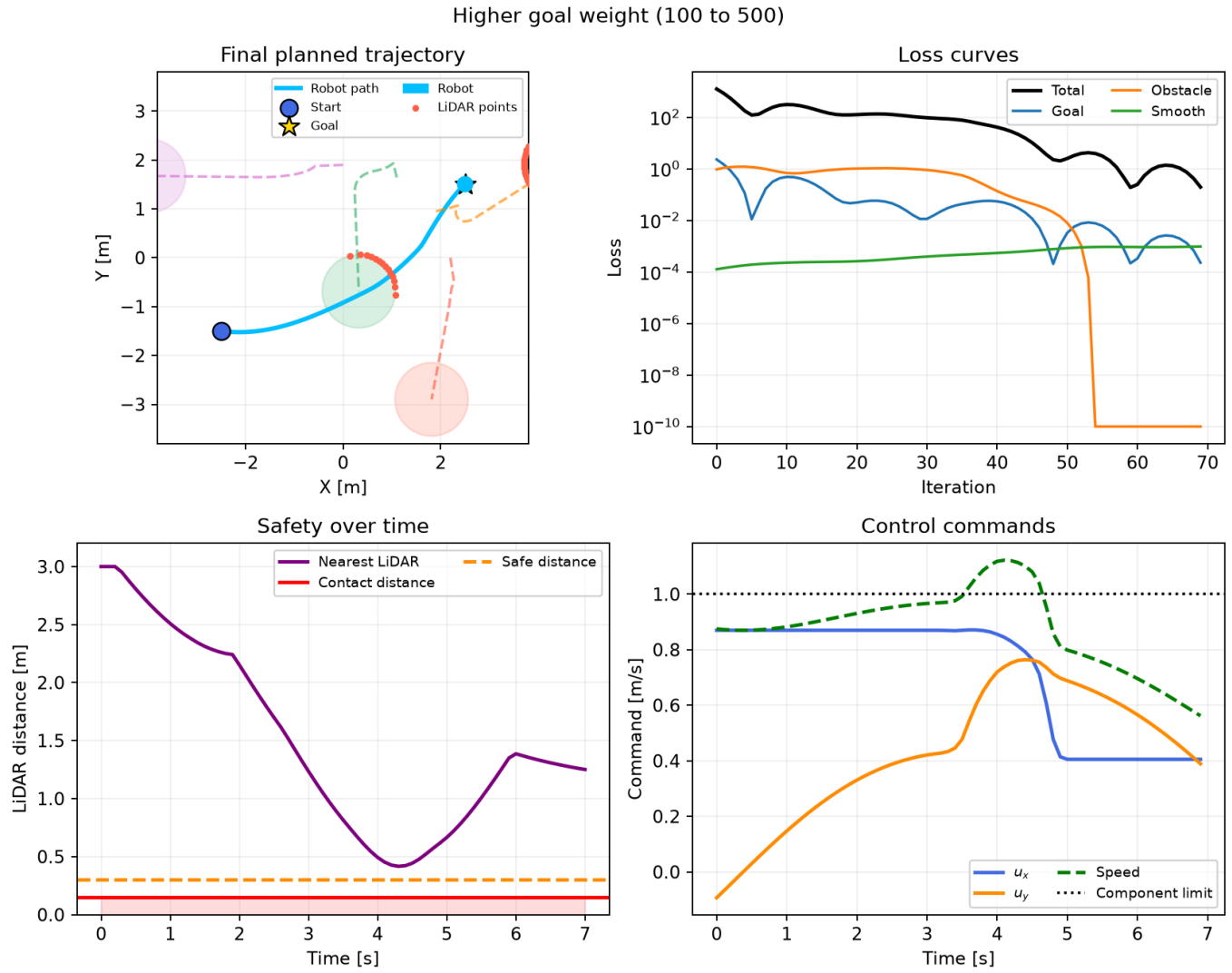


图 8-8：终点精度提高，同时路径与障碍物更接近。

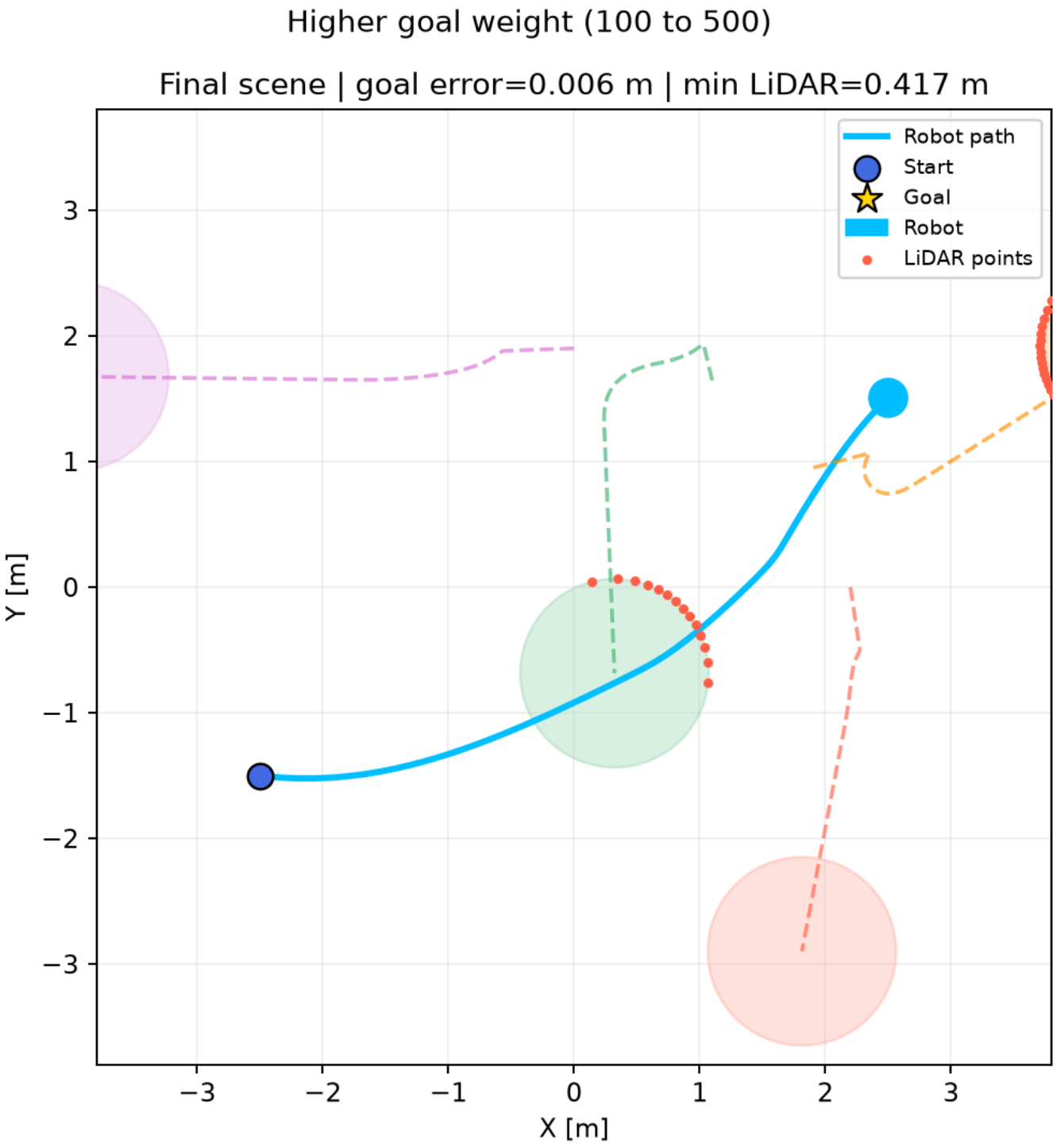


图 8-9：机器人最终位置几乎与目标重合。

结论：提高 Goal 权重能够提高本次实验的终点精度，但会压缩安全余量；继续增大不一定继续改善，因为轨迹优化是非凸的。

8.4 实验四：避障权重降低到 10

避障权重降低后，最小 LiDAR 距离仅为 `0.0242 m`，远低于 `0.30 m` 安全线，也低于 `0.15 m` 接触线。共有 10 个时间步违反安全要求，其中 6 个时间步发生实体接触。Obstacle Loss 为 `0.3998`。

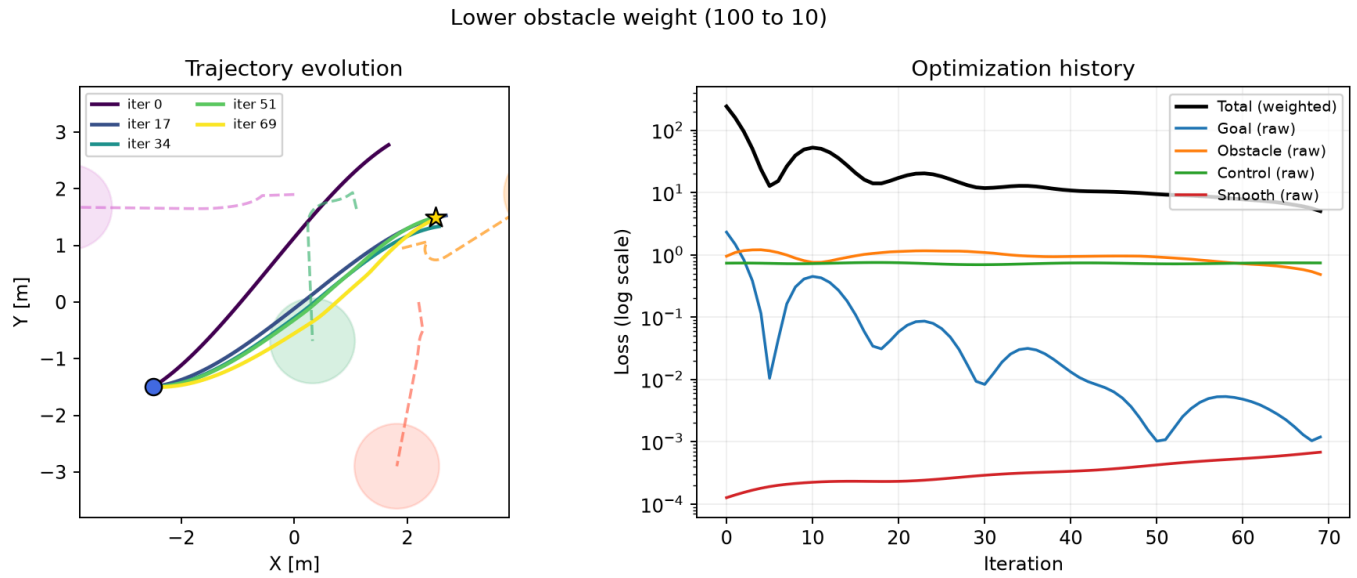


图 8-10：避障梯度过弱，轨迹更容易穿入障碍物区域。

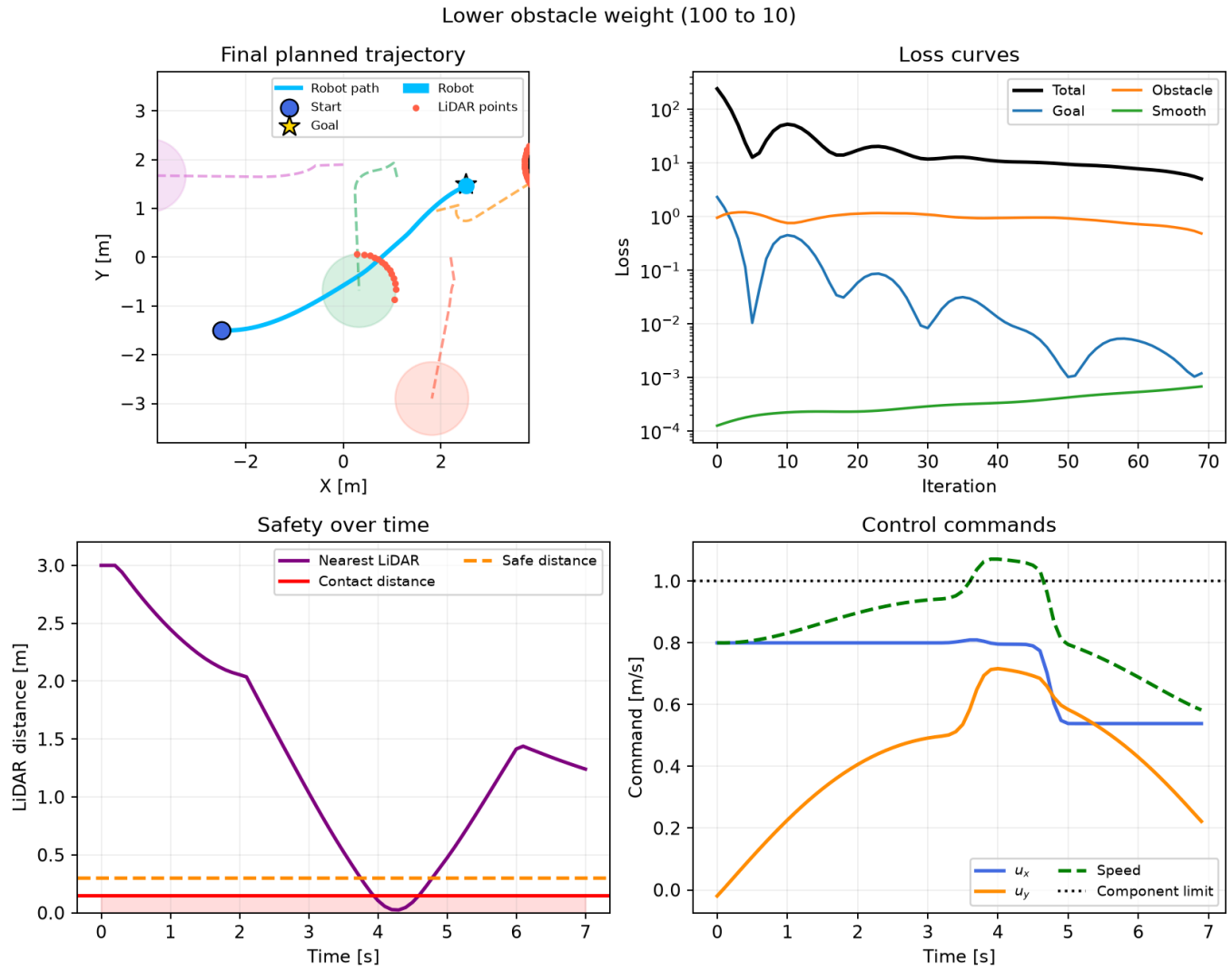


图 8-11：安全距离曲线明显跌破安全线和接触线。

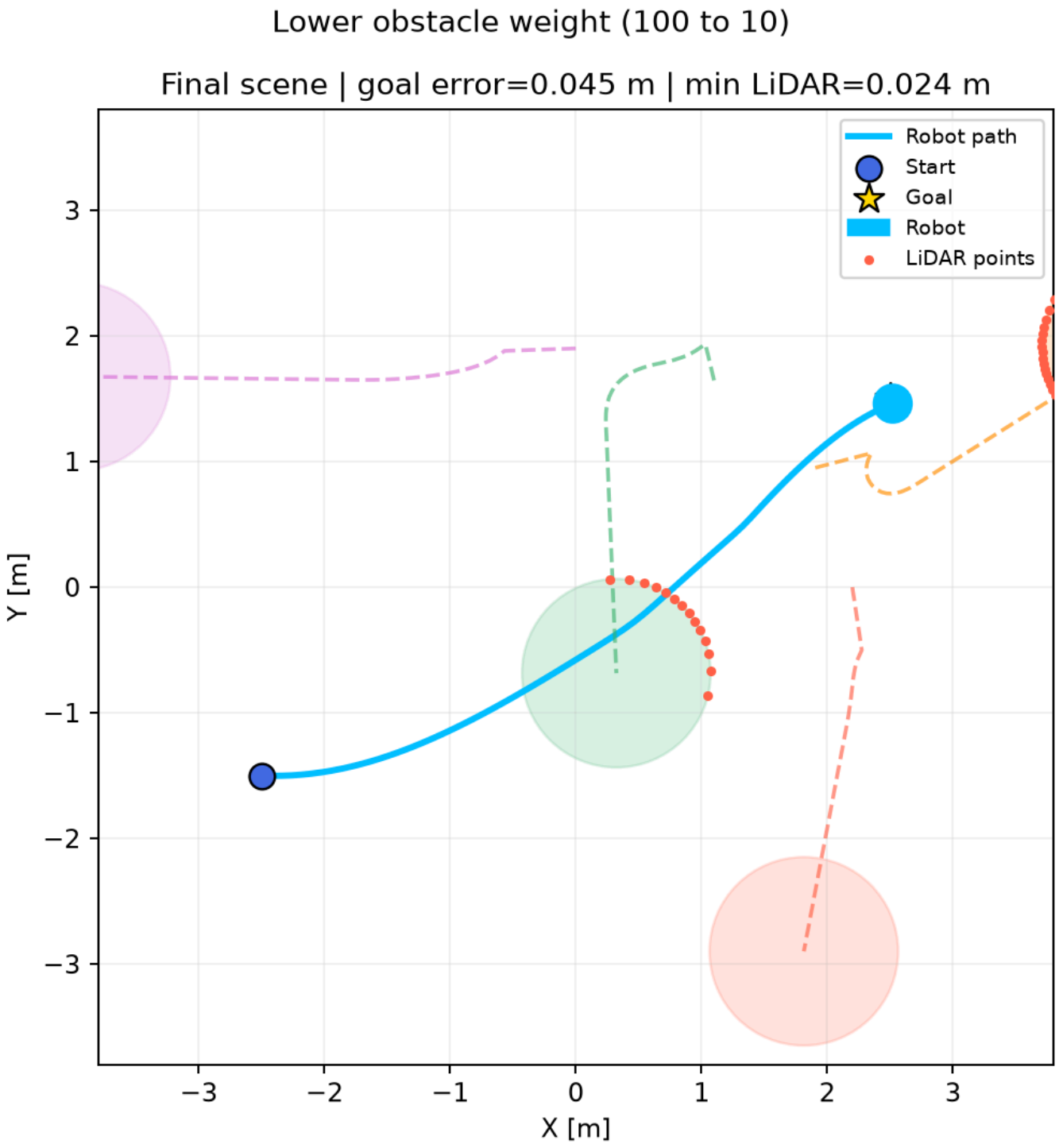


图 8-12：低避障权重组的最终空间状态。碰撞发生在运动过程中的某些时刻，因此只看最终帧可能看不出来，必须同时查看安全距离曲线。

结论：避障权重过低会导致碰撞。即使目标误差不大，这条路径也必须判定为失败。

8.5 实验五：避障权重提高到 800

提高避障权重后，Obstacle Loss 为 0，轨迹没有进入安全区。最小 LiDAR 距离为 `0.3412 m`，只比安全线高 `0.0412 m`。路径长度增加到 `6.8562 m`，相邻控制变化指标增加到 `0.05904`。

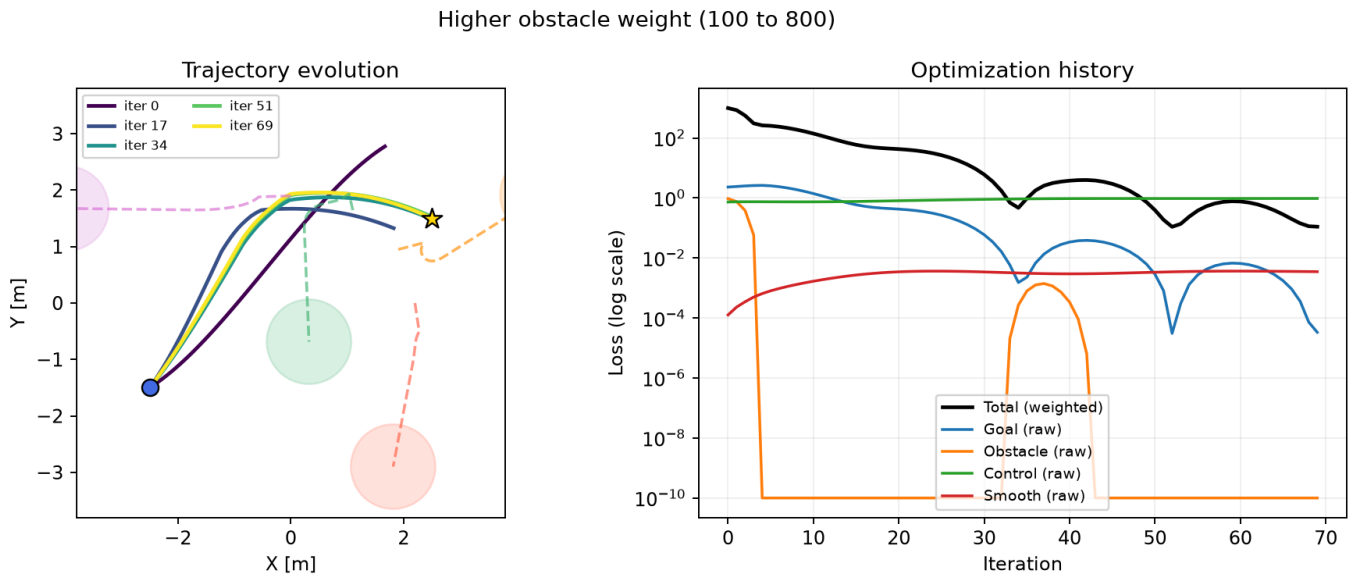


图 8-13：高避障权重改变了优化过程和最终绕行路线。

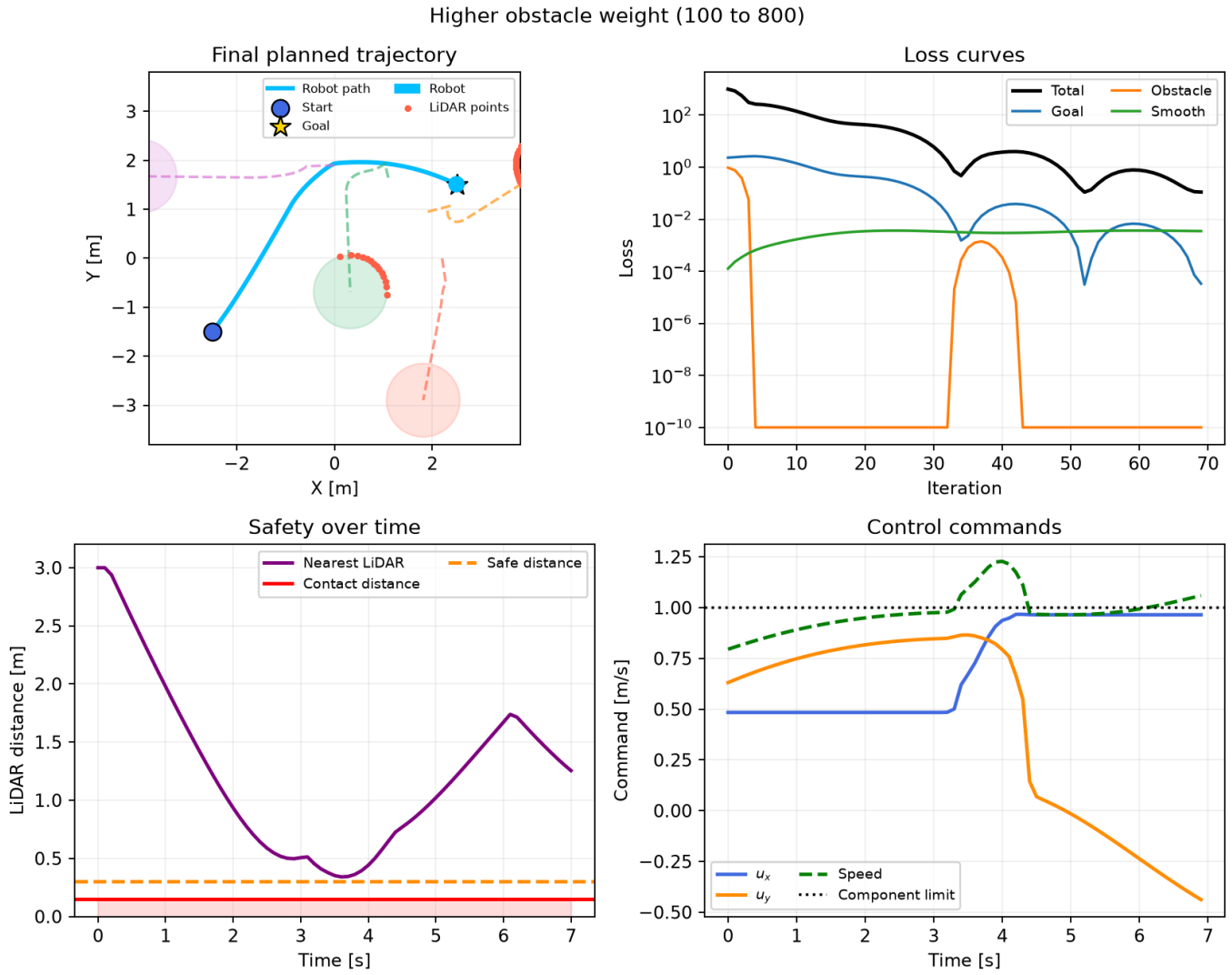


图 8-14：轨迹保持在安全线外，但路径和控制变化均有所增加。

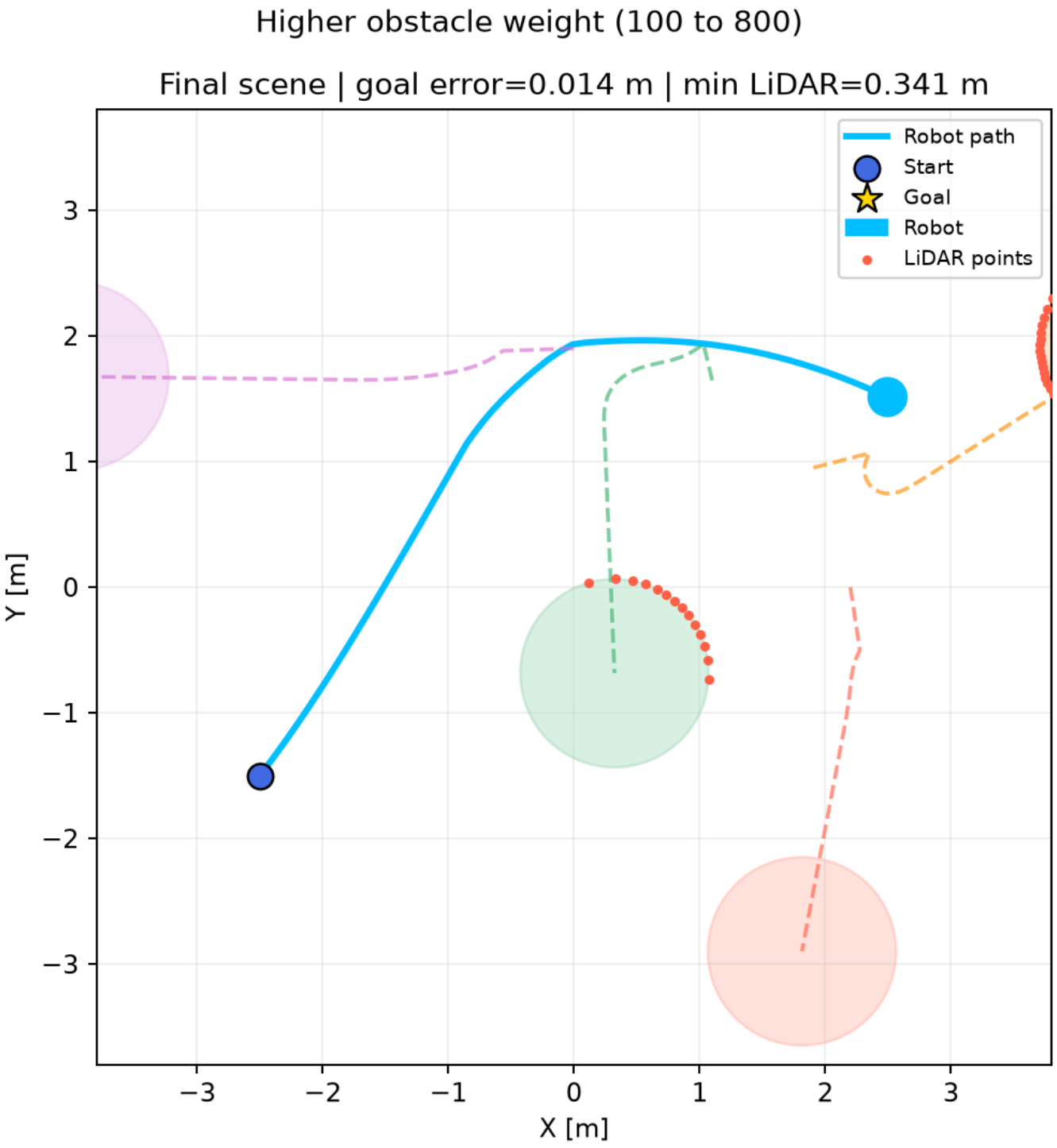


图 8-15：高避障权重组的最终状态。

值得注意的是，该组最小距离反而小于基准组的 `0.6241 m`。原因有两个：

1. 避障函数只惩罚 $d < 0.30\ \mathrm{m}$ 的状态，一旦越过安全线，继续增大距离没有奖励；
2. 该问题是非凸优化，提高权重可能使优化器进入另一条局部最优路径。

结论：提高 Obstacle 权重可以增强"不能进入危险区"的优先级，但不能保证最小距离按权重单调增加。若要求更大的物理间隔，应提高 `LIDAR_SAFE_DISTANCE`，而不仅是提高权重。

8.6 实验六：移除控制损失 `(100, 100, 0, 3)`

移除 Control Loss 后，目标误差、最小距离、路径长度、平均速度和控制变化均与基准组几乎相同。这表明当前 `0.10` 的控制权重在本场景中的影响很弱。

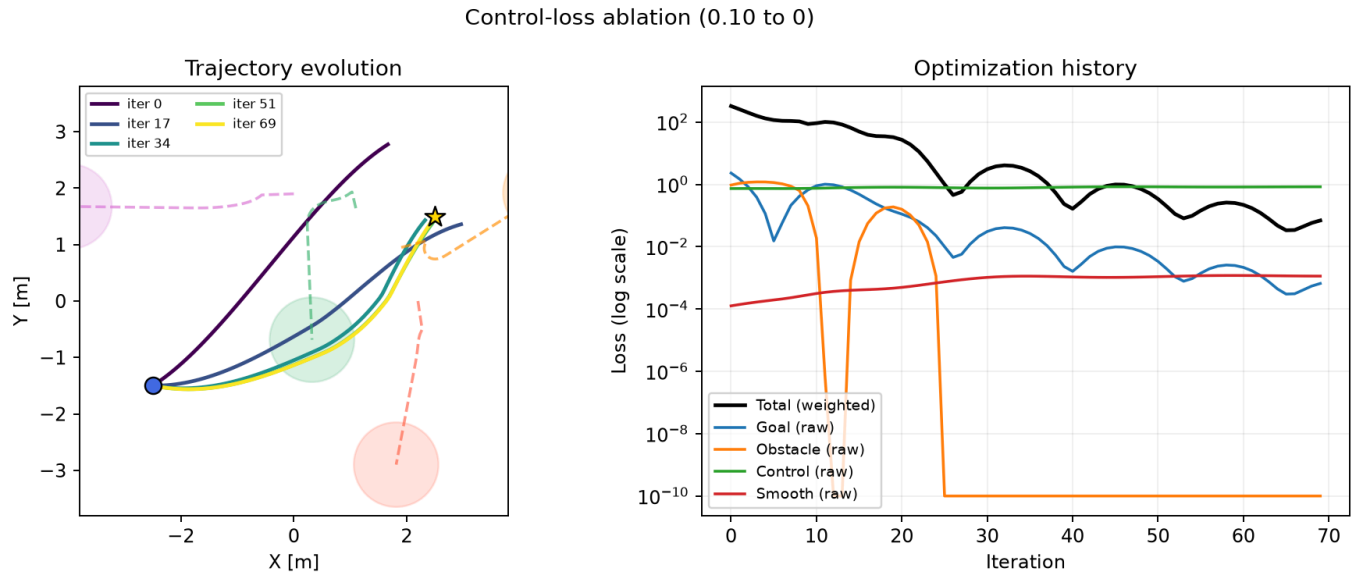


图 8-16：移除控制项后的轨迹和 Loss。

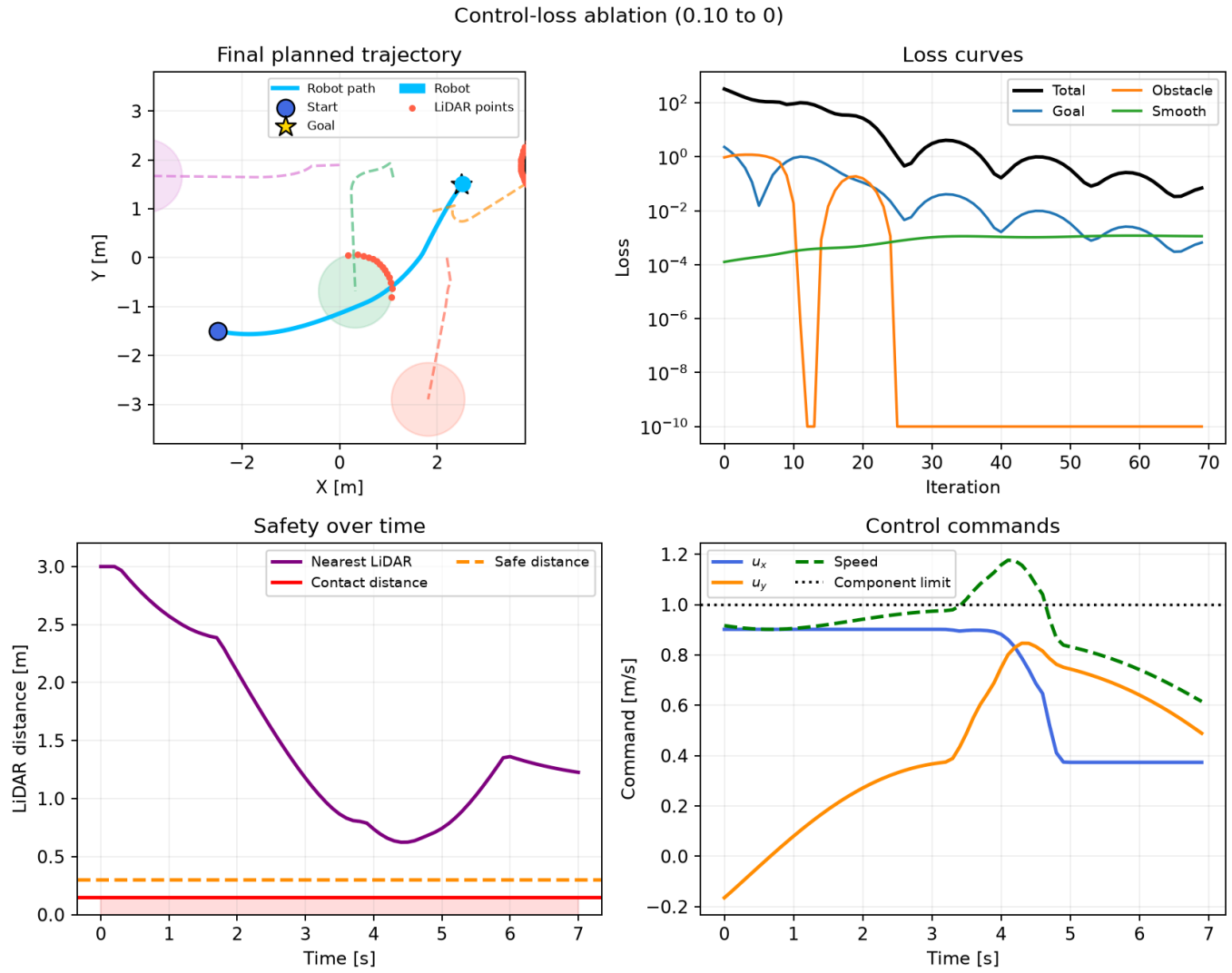


图 8-17：控制曲线与基准组基本重合。

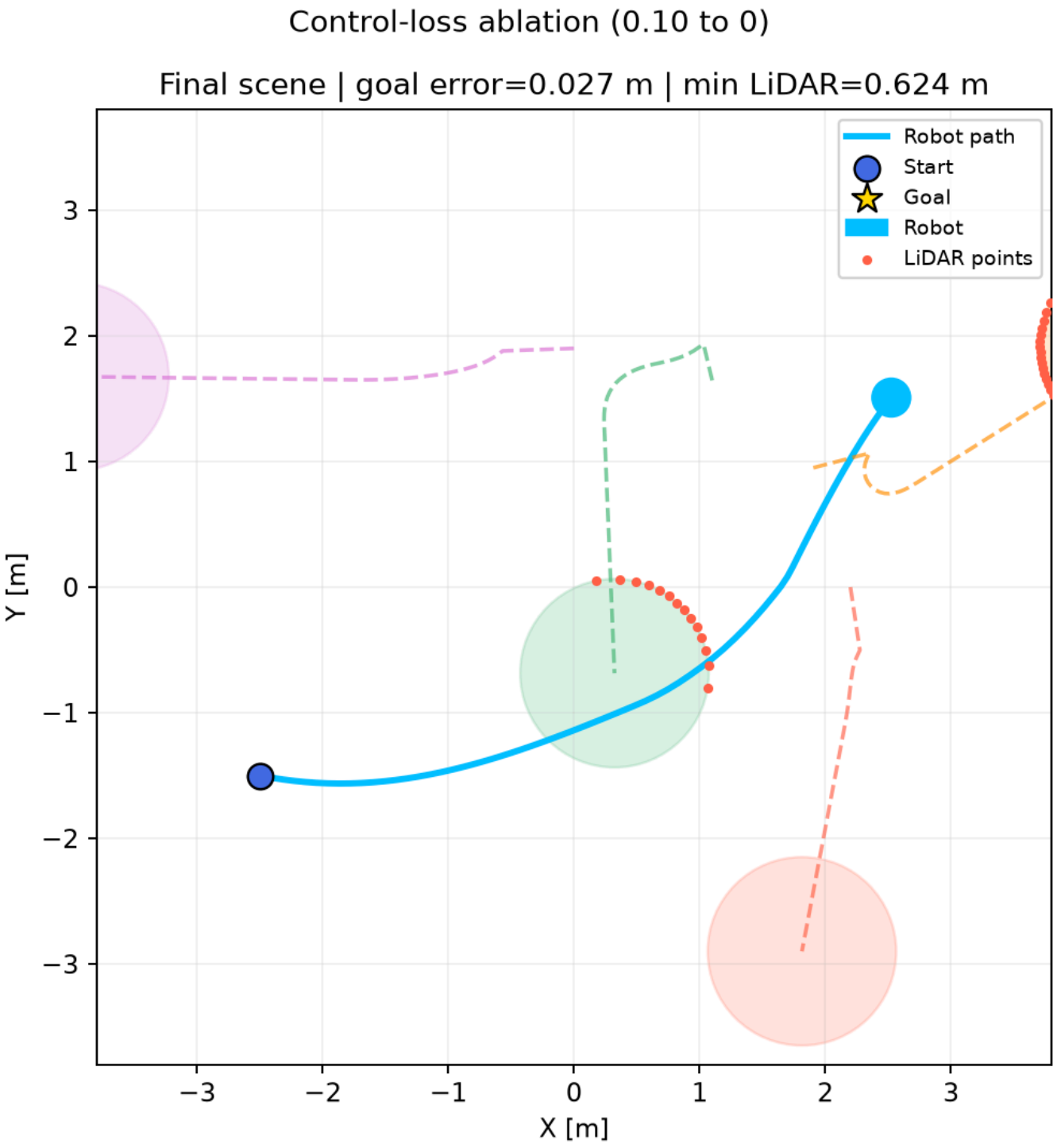


图 8-18：移除控制项后的最终场景。

结论：默认 `Control = 0.10` 主要起轻微正则化作用，而不是决定路径形状的主要因素。

8.7 实验七：控制权重提高到 100

强控制约束使 Control Loss 从 `0.8398` 降至 `0.7400`，平均速度从 `0.9067 m/s` 降至 `0.8546 m/s`。但最终目标误差增加到 `0.0853 m`，说明节省控制量会牺牲到达精度。

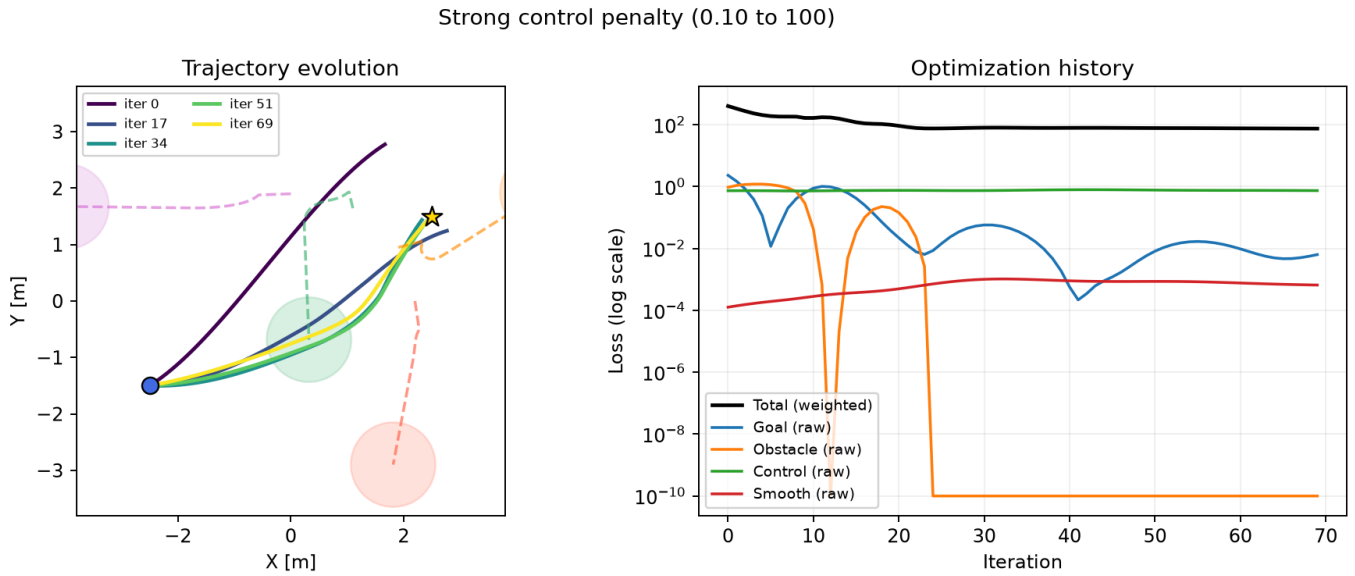


图 8-19：高控制权重使控制能量在总目标中占主导。

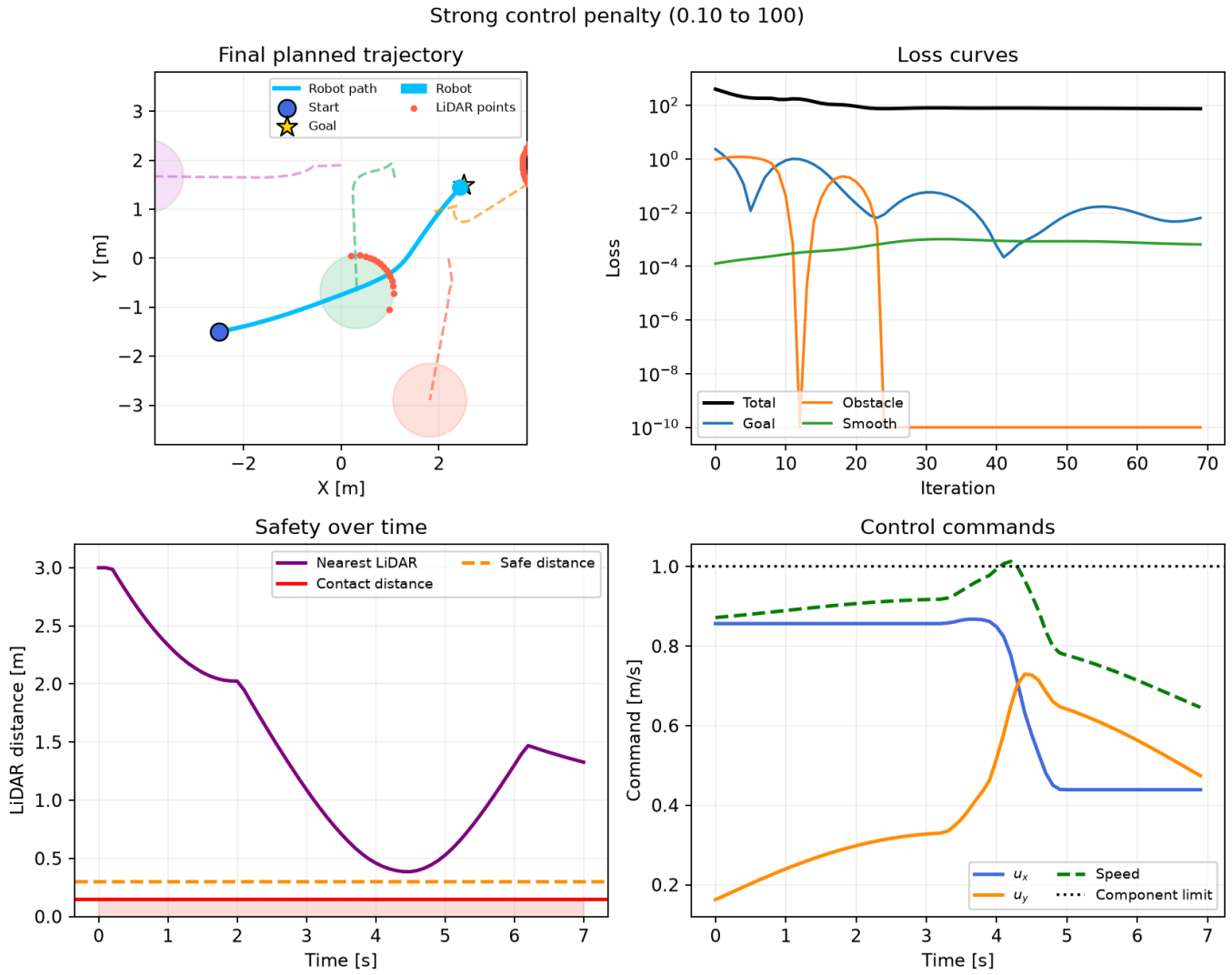


图 8-20：平均速度和控制变化下降，但终点误差增大。

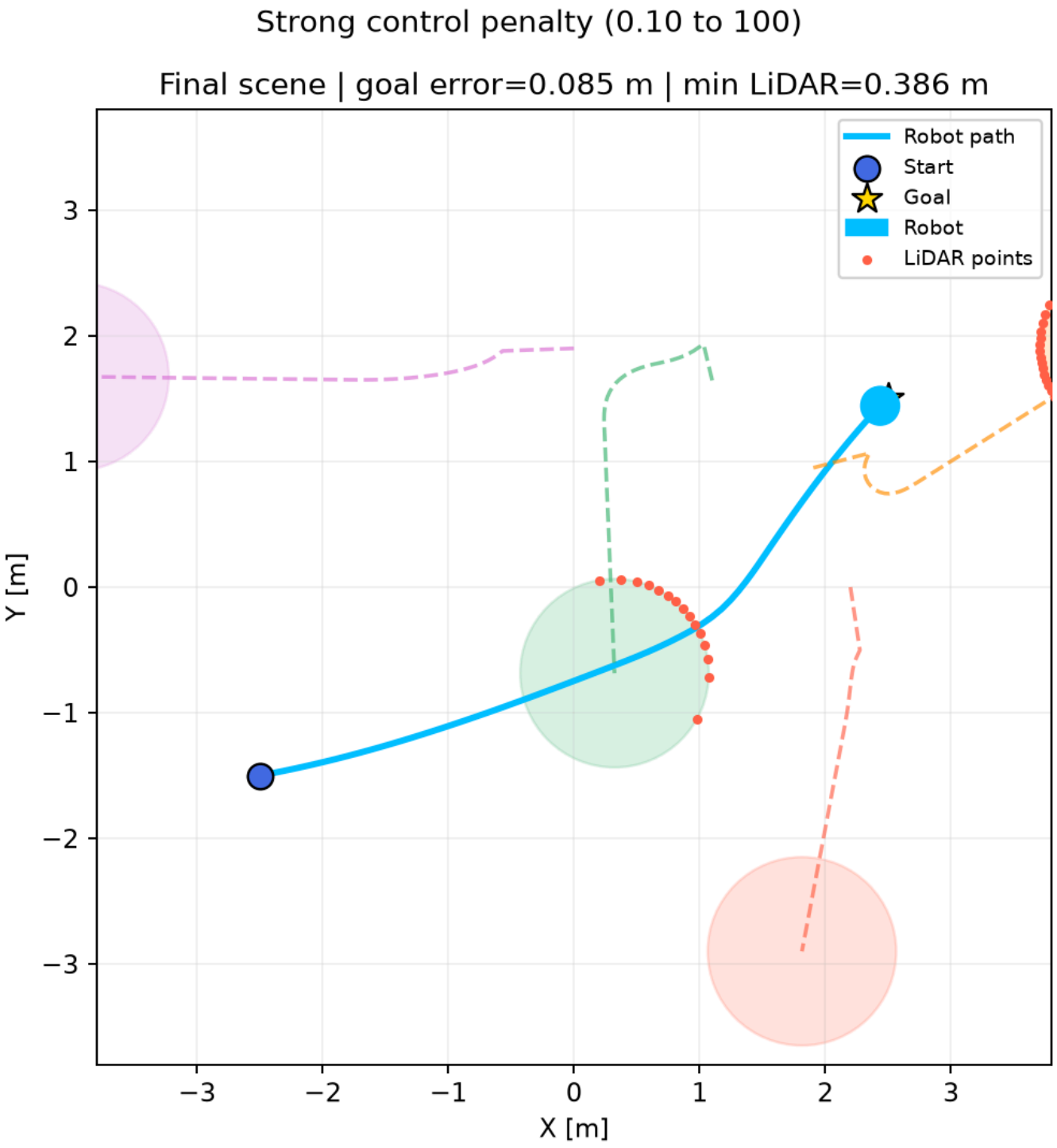


图 8-21：强控制约束下的最终位置。

该组 Total Loss 为 `74.7300`，主要原因是约 `0.7400` 的 Control Loss 被乘以 100。它不能与基准组的 Total Loss 直接比较。

结论：Control 权重体现"到达性能"和"控制能量"之间的交换，过大会使机器人不愿充分运动。

8.8 实验八：移除平滑损失 (100, 100, 0.10, 0)

移除 Smooth Loss 后，各项物理指标与基准组几乎相同，只有 Δu_{RMS} 从 0.03377 轻微增加到 0.03396。这说明默认权重 3 在当前场景下同样属于较弱的正则项。

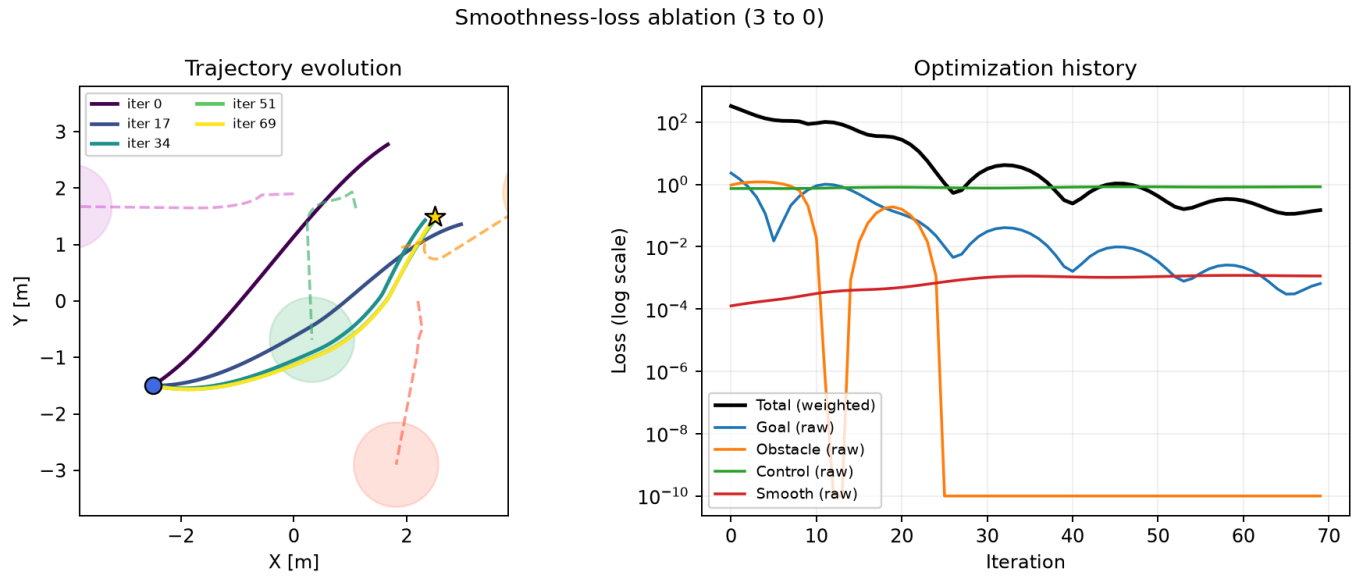


图 8-22：移除平滑项后的轨迹演化。

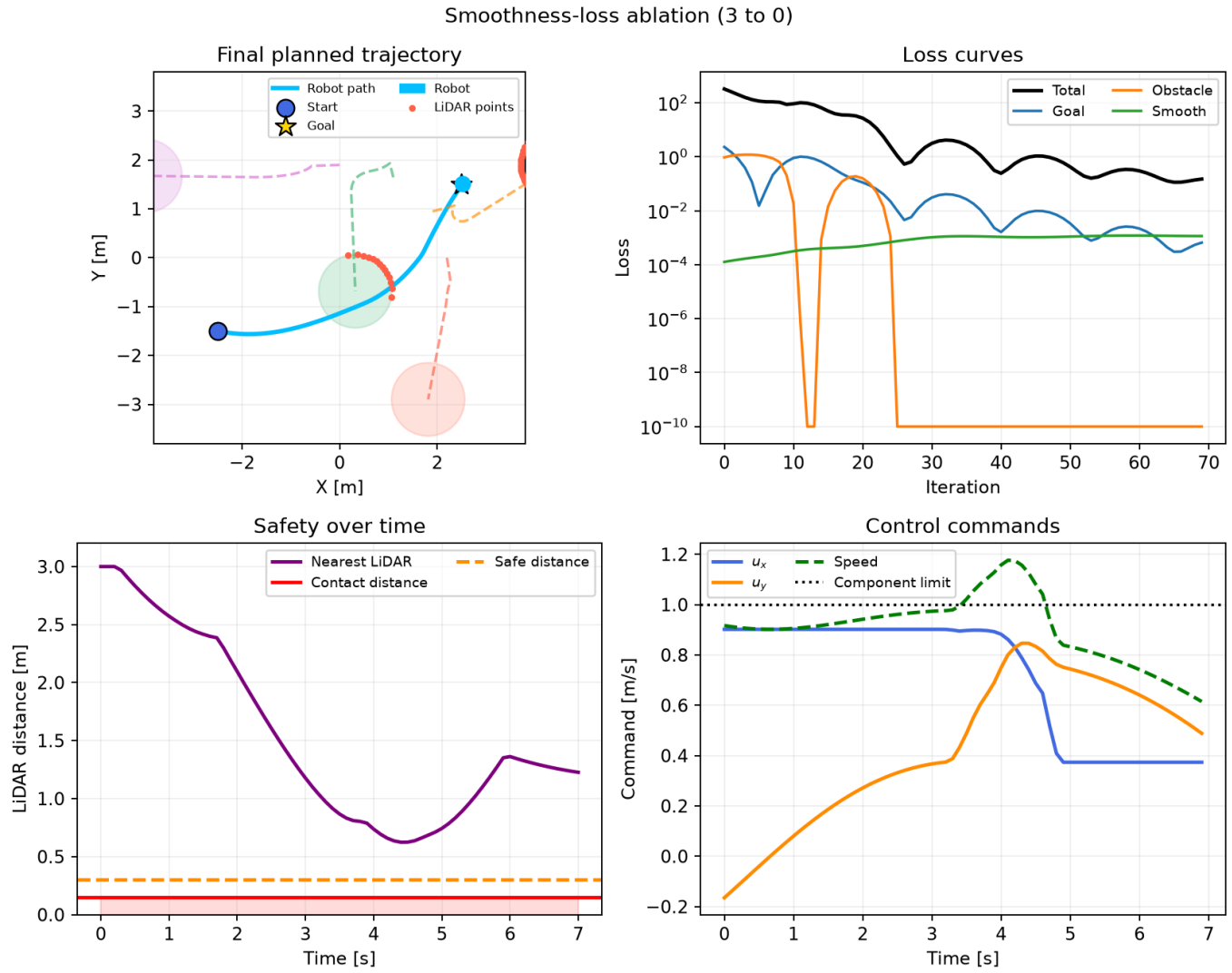


图 8-23：控制曲线只出现非常轻微的变化。

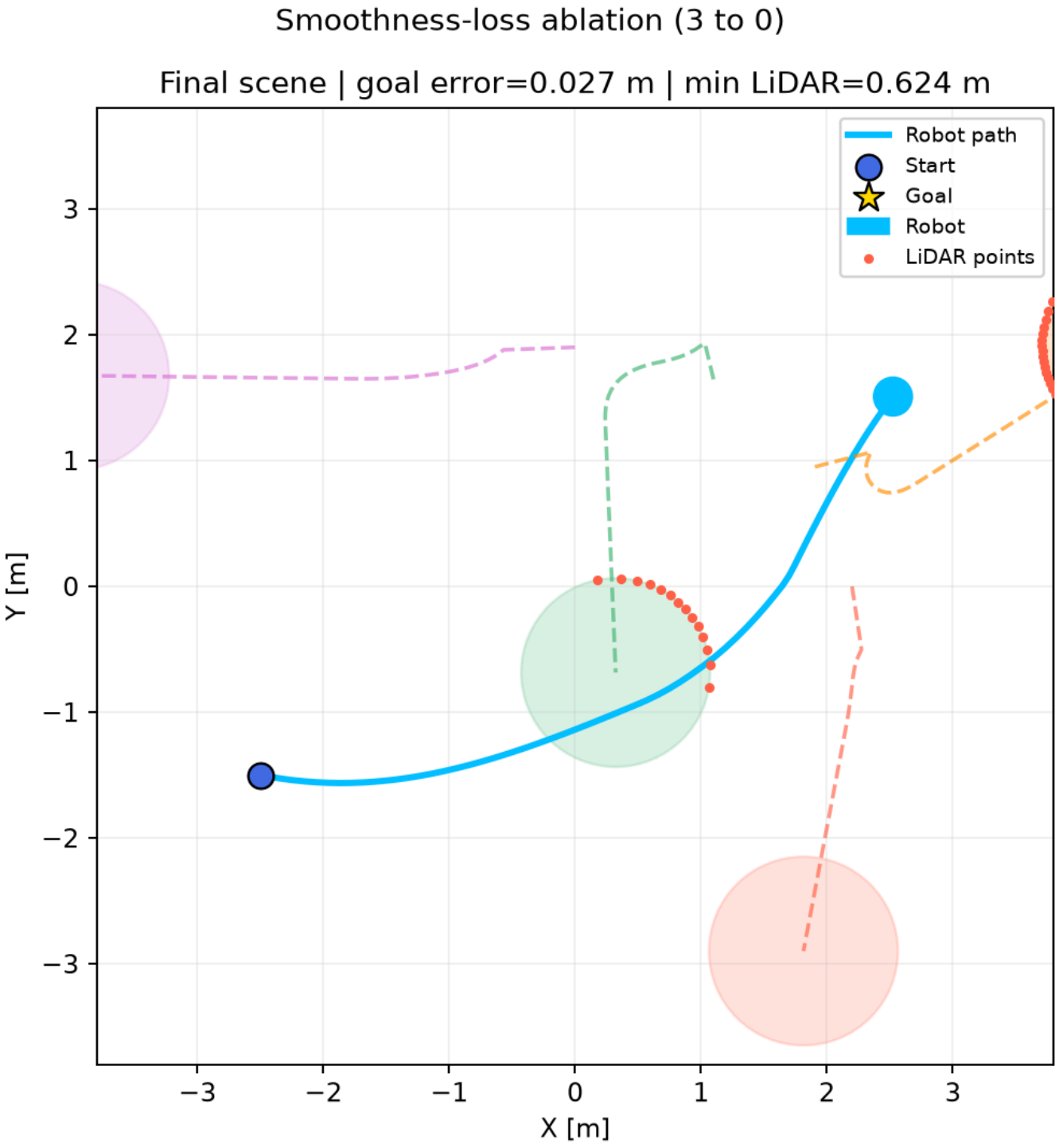


图 8-24：移除平滑项后的最终场景。

结论：默认 Smooth 权重对当前结果影响有限，但在更复杂、需要快速转向的环境中可能更重要。

8.9 实验九：平滑权重提高到 3000

强平滑约束使原始 Smooth Loss 从 `1.140×10⁻³` 降至 `4.933×10⁻⁴`，下降约 56.7%；Δu_{RMS} 从 `0.03377` 降至 `0.02221`，下降约 34.2%。与此同时，目标误差从 `0.0268 m` 略微增加到 `0.0307 m`。

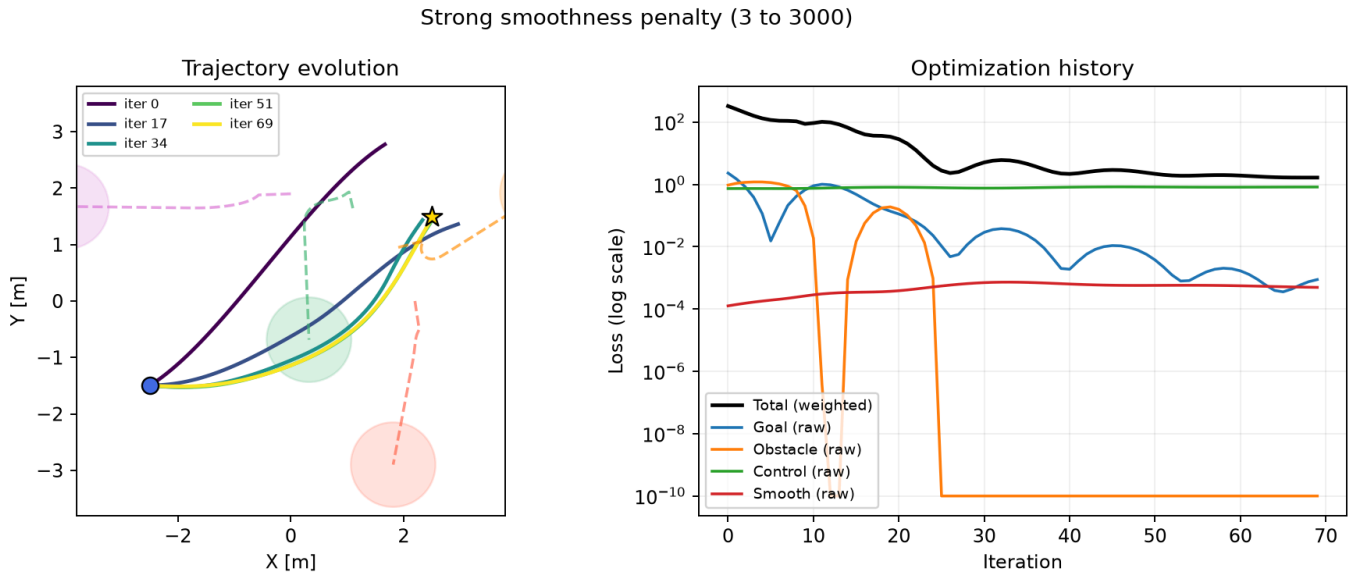


图 8-25：高平滑权重使相邻控制量变化受到更强约束。

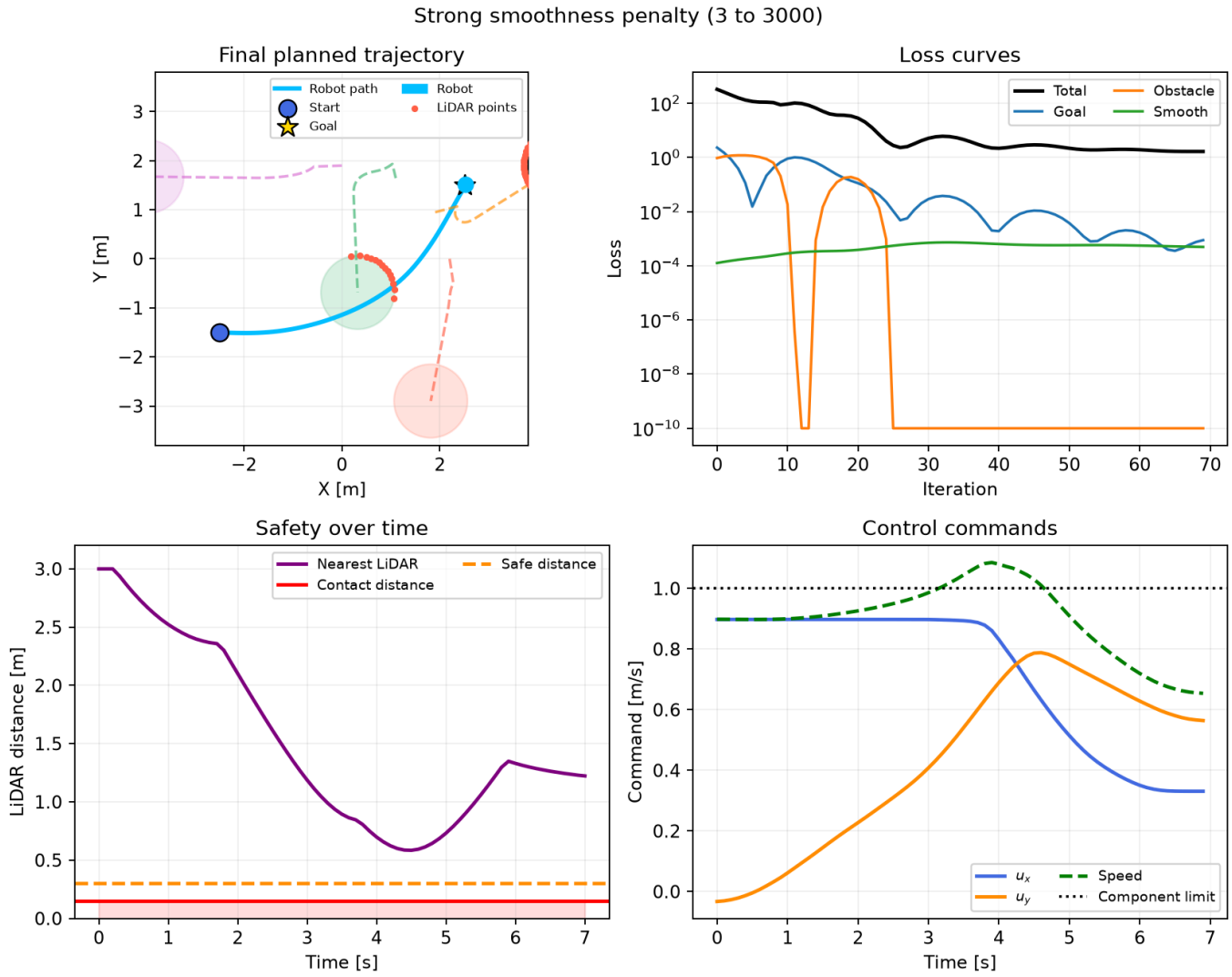


图 8-26：控制曲线更平缓，同时目标精度略有下降。

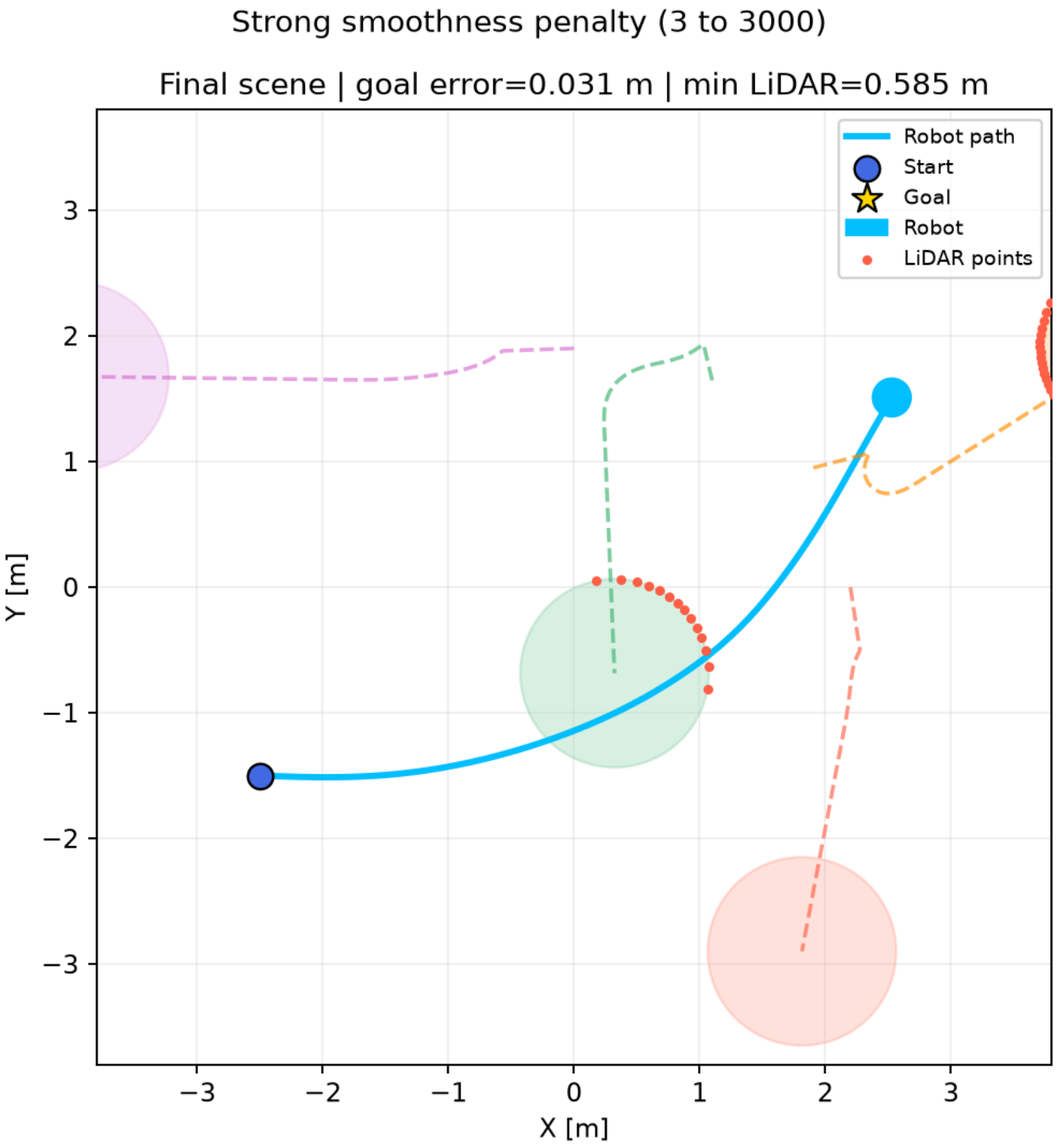


图 8-27：强平滑约束下的最终场景。

结论：提高 Smooth 权重确实可以减少速度变化，但过强时会降低轨迹调整能力，并可能牺牲目标精度。

---

9. 综合结论

9.1 四项 Loss 各自承担不可完全替代的任务

1. **Goal Loss 决定是否完成导航任务。** 移除后即使 Total Loss 很低，机器人仍会停在离目标约 `3.15 m` 的位置。

2. **Obstacle Loss 决定是否满足安全要求。** 权重从 100 降到 10 后出现 10 个不安全时间步和 6 个接触时间步。
3. **Control Loss 调节运动消耗。** 当前 `0.10` 的影响很弱；提高到 100 后控制能量下降，但目标误差明显增加。
4. **Smooth Loss 调节控制连续性。** 当前 `3` 的影响很弱；提高到 3000 后速度变化明显减小，但目标精度略有损失。

9.2 最小 Total Loss 不等于最好路径

当权重保持不变时，总 Loss 越小通常表示优化在进步。但改变权重后，总公式的尺度也改变了，不能跨实验直接比较 Total Loss。

最典型的例子是：

- 基准组 Total Loss：`0.1591`，目标误差：`0.0268 m`；
- 移除目标项 Total Loss：`0.0524`，目标误差：`3.1538 m`。

后者的 Total Loss 更小，却明显没有完成任务。

9.3 应优先使用物理指标评价结果

推荐按以下顺序判断：

1. 是否出现碰撞，即 $d_{\min} < 0.15\ \mathrm{m}$；
2. 是否违反安全线，即 $d_{\min} < 0.30\ \mathrm{m}$；
3. 最终目标误差是否足够小；
4. 在安全到达的前提下，再比较路径长度、平均速度和控制平滑性。

9.4 权重调整不一定产生单调结果

动态障碍物、LiDAR 几何计算、`ReLU` 分段函数和长时域控制共同构成了一个非凸优化问题。因此：

- 增大某个权重不保证对应物理指标严格单调改善；
- 优化器可能进入不同的局部最优路径；
- 应通过多组实验和物理指标验证，而不能只依赖理论直觉。

9.5 对当前代码的建议

当前基准权重 `(100, 100, 0.10, 3)` 已经能得到安全且准确的路径。如果更重视终点精度，可以尝试适度提高 Goal 权重，但必须同时检查最小安全距离。如果希望机器人始终离障碍物更远，优先提高：

```
LIDAR_SAFE_DISTANCE
```

或其中的：

```
SAFETY_MARGIN
```

而不是只增加 Obstacle 权重。因为当前避障函数在超过安全线后就变为 0，不会继续奖励更大的距离。

10. 终端输出的正确解释

原程序终端显示类似：

```
70/70 [..., distance=0.590, loss=0.297]
```

其中：

- `distance` 是轨迹中的最小 LiDAR 障碍距离，不是到目标点的距离；
- `loss` 是加权总 Loss；
- 代码只在第 `0、20、40、60` 次迭代更新进度条后缀，所以 `70/70` 后保留的数字实际上可能仍是第 60 次迭代的值，而不是最终第 69 次结果。

本文所有表格均在 70 次优化结束后重新模拟并计算最终指标。因此基准组报告值为：

```
最终目标误差      = 0.0268 m
最终最小LiDAR距离 = 0.6241 m
最终安全余量      = 0.3241 m
最终Total Loss    = 0.1591
```

这些数值比终端进度条最后保留的显示更适合用于正式汇报。

11. 可复现实验文件

- `run_loss_weight_experiments.py`：批量运行 9 组实验并生成图片；
- `experiment_metrics.csv`：全部量化指标，便于使用 Excel 继续分析；
- `experiment_metrics.json`：机器可读的完整指标；
- `figures/`：9 组实验共 27 张正式图片。

实验脚本不会修改桌面上的原始路径规划代码，只读取其中的机器人模型、障碍物模型和 LiDAR 模型，然后使用不同权重重新计算 Loss。